

Algoritmer

Dijkstra & Astar / A*



Agenda

- Hvad er pathfinding
- Dijkstra algoritmen
- Øvelse
- A* Algoritmen – **12:00**
 - Eksempel
 - Opsummering
- Øvelse





Hvad er pathfinding



Hvad er rutebestemmelse

- Muligheden for at beregne ruten fra punkt A til punkt B
 - Finde den korteste rute (Shortest path)
 - Få en AI til at bevæge sig korrekt / imod et punkt
- Mest anvendte pathfinding algoritmer indenfor spil og udvikling generelt er:
 - Dijkstra's algoritme
 - Finder den korteste vej til alle andre noder i grafen.
 - Kan termineres tidligere, hvis kun ruten til et punkt er vigtig.
 - A*
 - En hurtigere form af Dijkstra's algoritme, der bruger heuristik til kun at løbe relevante noder igennem – finder ikke nødvendigvis vejen til alle.
- Note: Selvom algoritmerne ofte anvendes på et grid, kan de sagtens anvendes på andre graftyper
 - Prisberegningerne skal blot laves dynamisk dvs. G og H. på astar og edge & weight i dijkstra
 - Et grid er vel også en graf? 😊



Dijkstra Algoritmen

Youtube video som forklare algoritmen
<https://www.youtube.com/watch?v=bZkzH5x0SKU>



Dijkstra

- Dijkstra algoritmen finder den hurtigste vej i en vægtet graf fra én node til én eller alle andre noder
 1. Den starter ved at sætte afstanden til alle andre noder til uendelig, undtagen for startstedet, hvor afstanden er 0.
 2. den undersøger derefter hver nærliggende node i startnoden og opdaterer afstanden til de noder, hvis der er en hurtigere vej.
 3. Trin 2 gentages, indtil alle noder er undersøgt eller mål er fundet.
 4. Dernæst kan den hurtigste vej findes ved at backtracke fra en slut node



Dijkstra

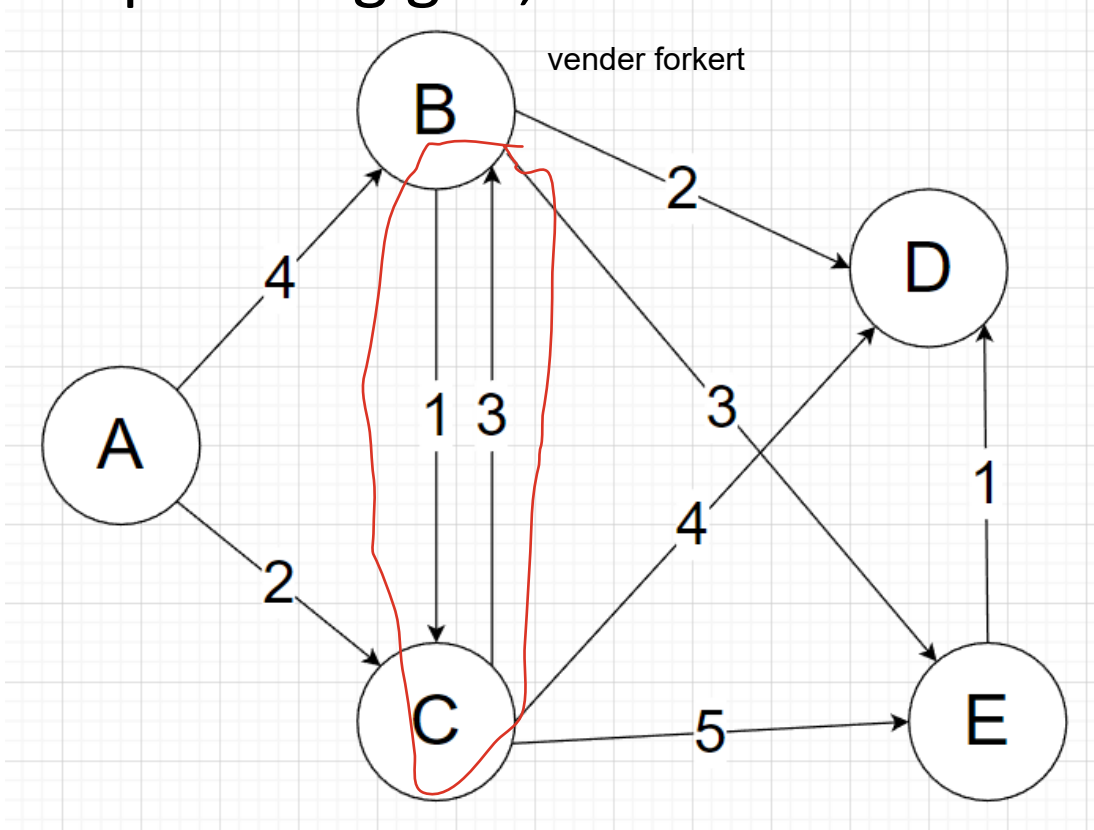
vores spilleplade

- `Input(startnode, directedgraph)` skal være `Input(startnode, slutnode, directedgraph)`
- Vi har brug for at holde styr på om noder er besøgte eller ej, samt hvad afstanden fra start noden til hver enkelt node er
- For at kunne finde den reelle path skal vi også have en parent i vores nodes
- Dette gør vi mindst har brug for 2 collections
 - Unvisited nodes{}
 - Distances {}



{ Dijkstra

- Oprindelig graf, sætter alle nodes til unvisited

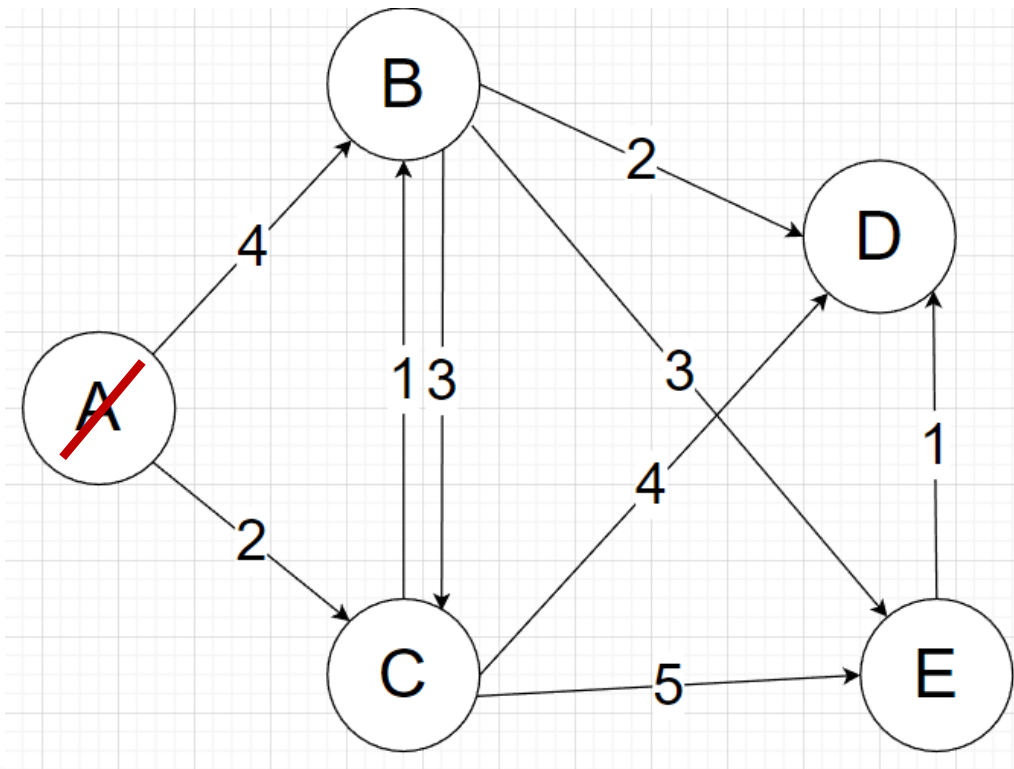


Unvisited nodes:
{A,B,C,D,E,F}



{ Dijkstra

- Sæt distance til startnode(A) til 0, og sæt alle andre distancer til ∞
- Vælg node med korteste distance, i dette tilfælde A (da vi sætter den til 0)
 - Vi kigger på alle unvisited nodes, og vælger den med lavest distance.
 - Fjerner A fra unvisited nodes



Distances(fra A):

A:0

B: ∞

C: ∞

D: ∞

E: ∞

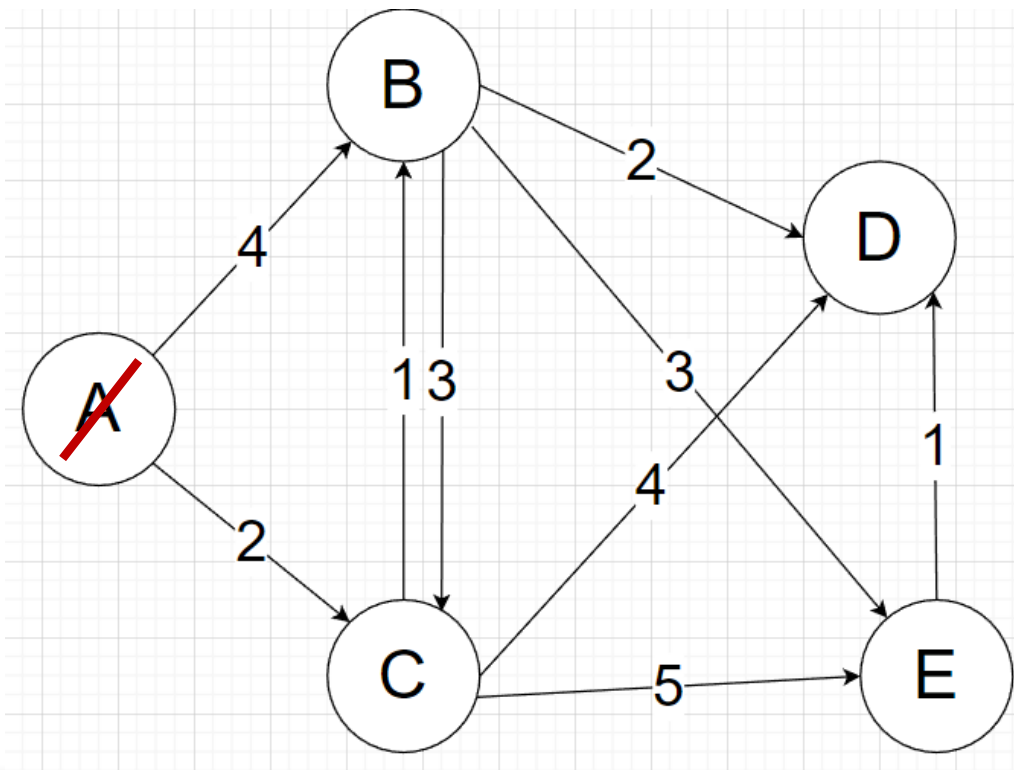
Unvisited nodes:

{~~A~~,B,C,D,E,F}



{ Dijkstra

- Undersøg noder vi kan nå fra A. Dette tilfælde B og C
- Opdater distances til disse, hvis de er mindre end hvad der er i distances



Distances(fra A):

A:0

B:4

C:2

D: ∞

E: ∞

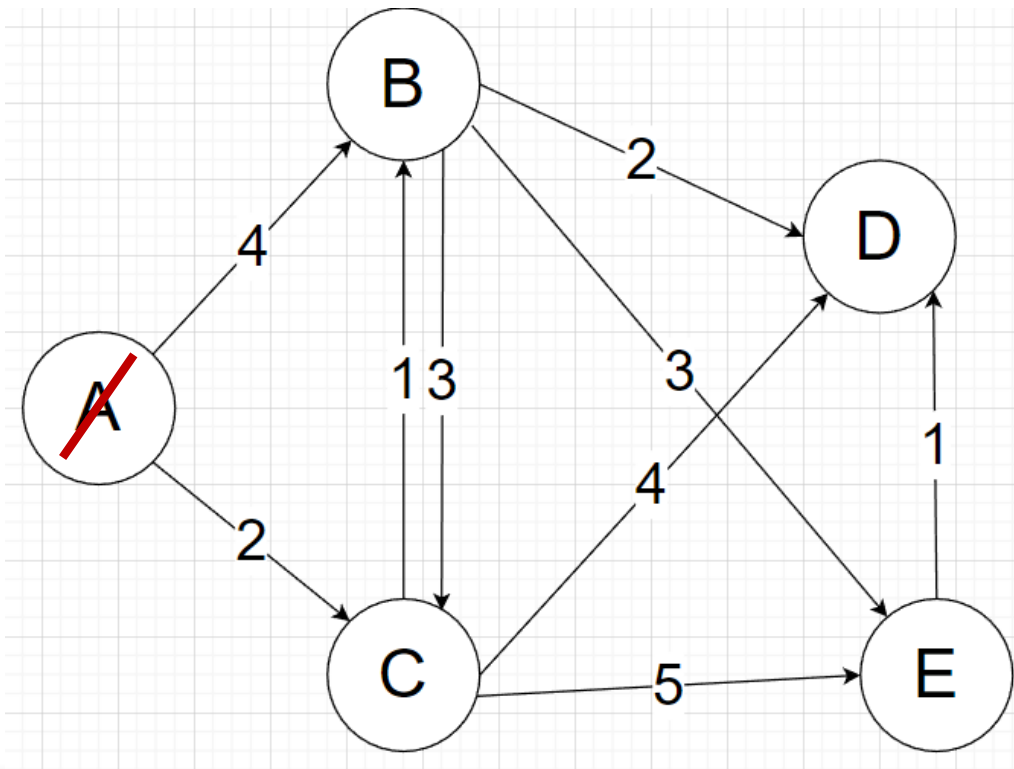
Unvisited nodes:

{~~A~~,B,C,D,E,F}



{ Dijkstra

- Gå til node, som ikke har været besøgt med den korteste distance
- I dette tilfælde C, som vi derfor rykker til



Distances(fra A):

A:0

B:4

C:2

D: ∞

E: ∞

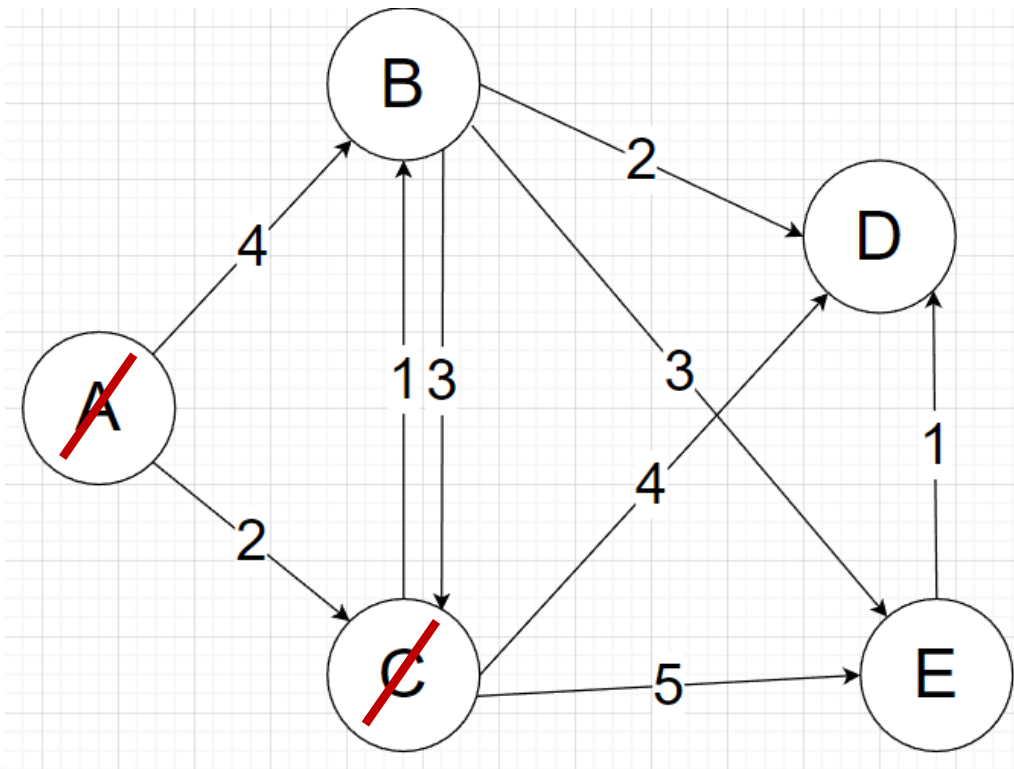
Unvisited nodes:

{~~A~~,B,C,D,E}



{ Dijkstra

- Fra C, undersøg igen kanter fra C, og opdater Distances
- Vi kan se at B nu har en hurtigere vej gennem C!



Distances(fra A):

A:0

B:3

C:2

D:6

E:7

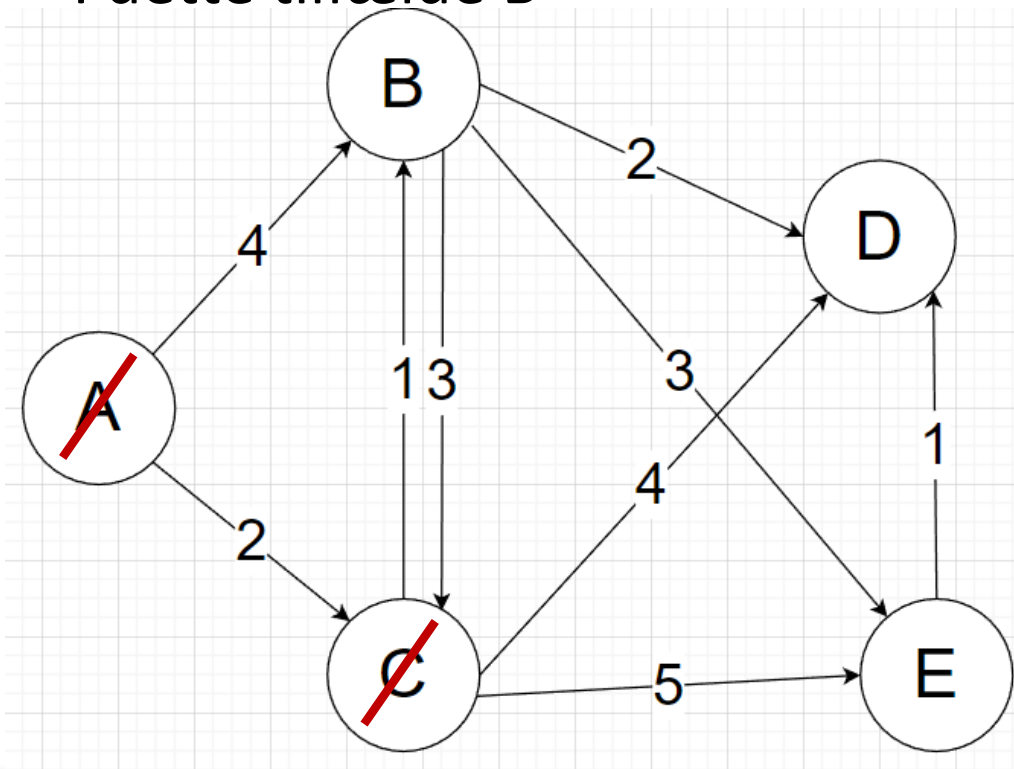
Unvisited nodes:

{~~A~~,B,~~C~~,D,E}



{ Dijkstra

- Igen vælger vi node med den korteste Distance, som vi endnu ikke har besøgt
- I dette tilfælde B



Distances(fra A):

A:0

B:3

C:2

D:6

E:7

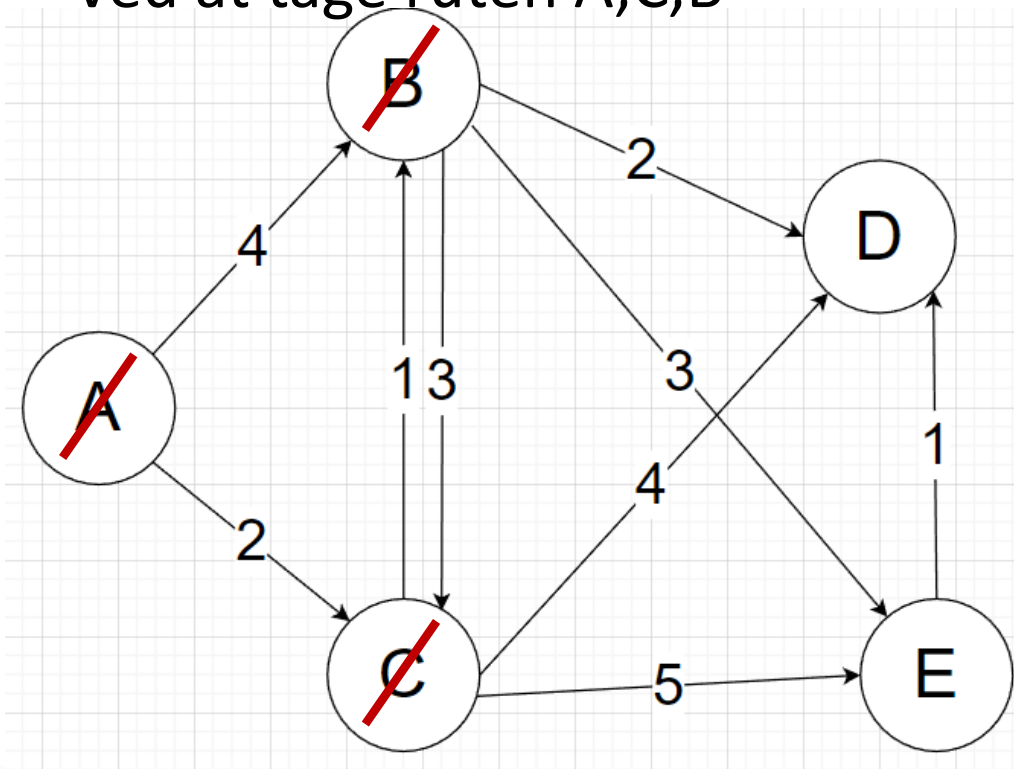
Unvisited nodes:

{~~A~~,B,~~C~~,D,E}



{ Dijkstra

- Fra B, undersøg igen kanter fra B, og opdater Distances
- Vi kan igen se at vi har fået en hurtigere vej til både D og E
- Ved at tage ruten A,C,B



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

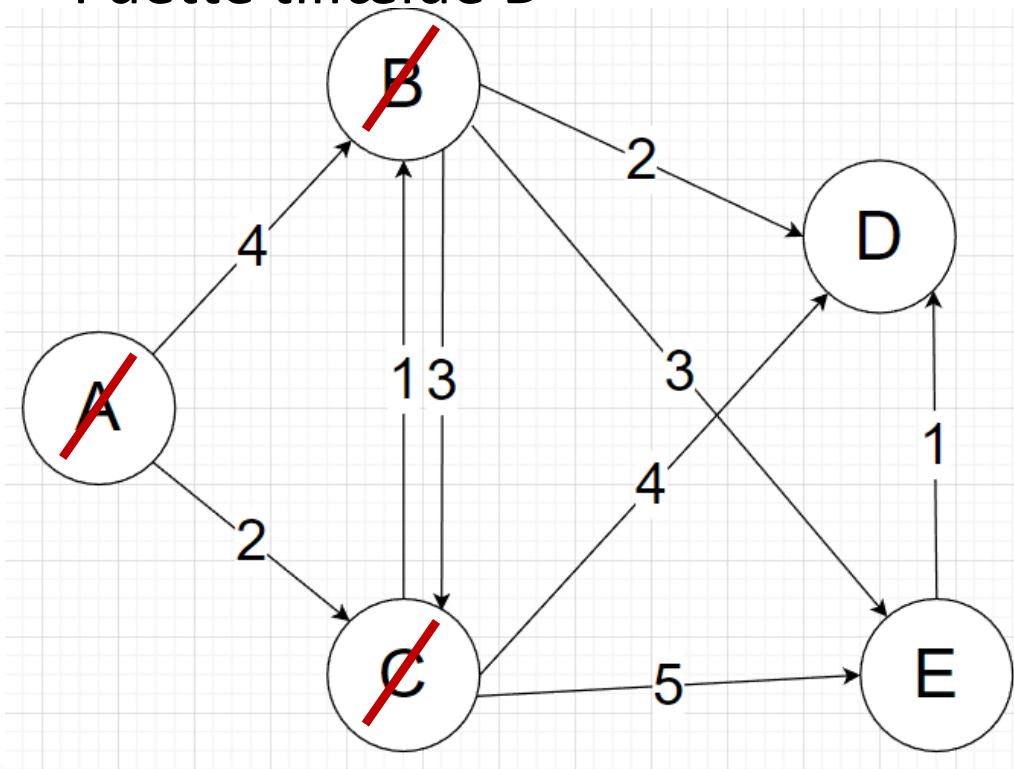
Unvisited nodes:

{~~A~~,~~B~~,~~C~~,D,E}



{ Dijkstra

- Igen vælger vi node med den korteste Distance, som vi endnu ikke har besøgt
- I dette tilfælde D



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

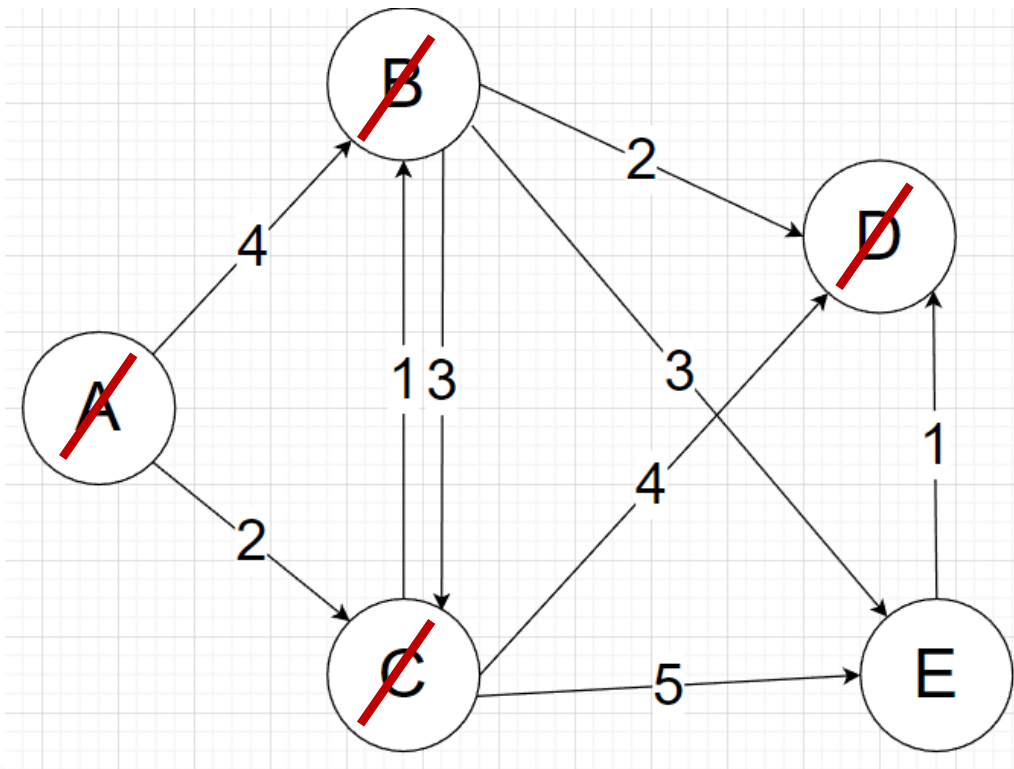
Unvisited nodes:

{~~A~~,~~B~~,~~C~~,D,E}



{ Dijkstra

- Denne gang er der ingen opdateringer til distances, da D ikke har nogen udadgående kanter



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

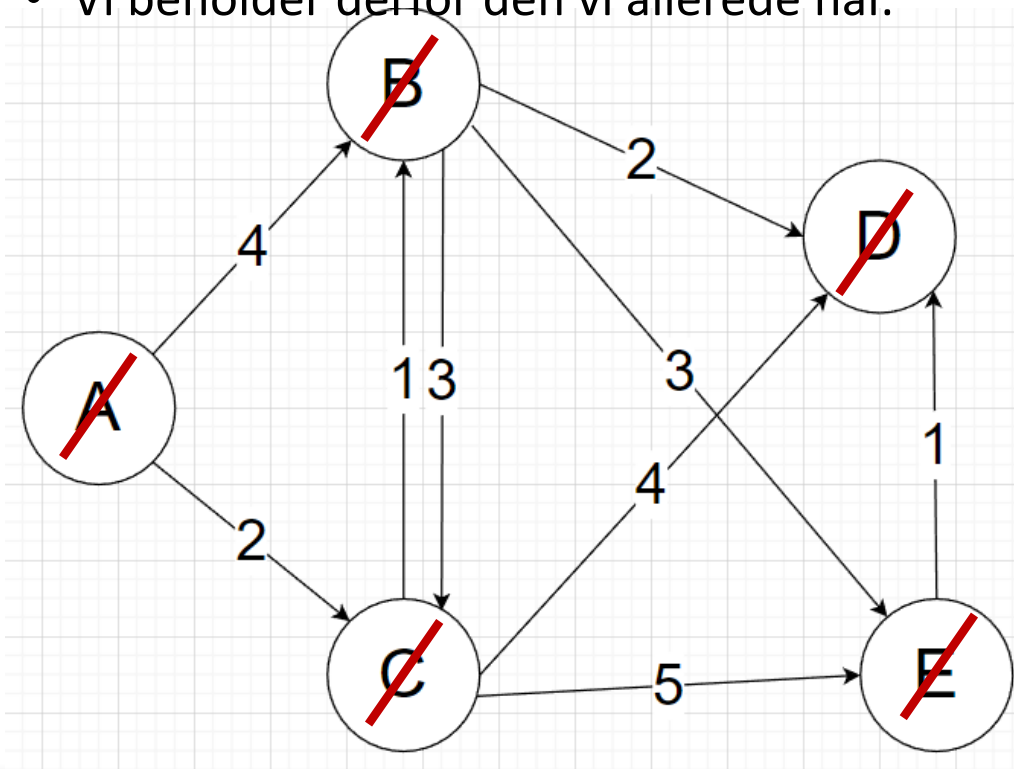
Unvisited nodes:

{~~A~~,~~B~~,~~C~~,~~D~~,E}



{ Dijkstra

- Derfor vælger vi E, korteste distance i unvisited
- Vi opdaterer Distances, men vejen er ikke hurtigere end den vi allerede havde til D
- Vi beholder derfor den vi allerede har.



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

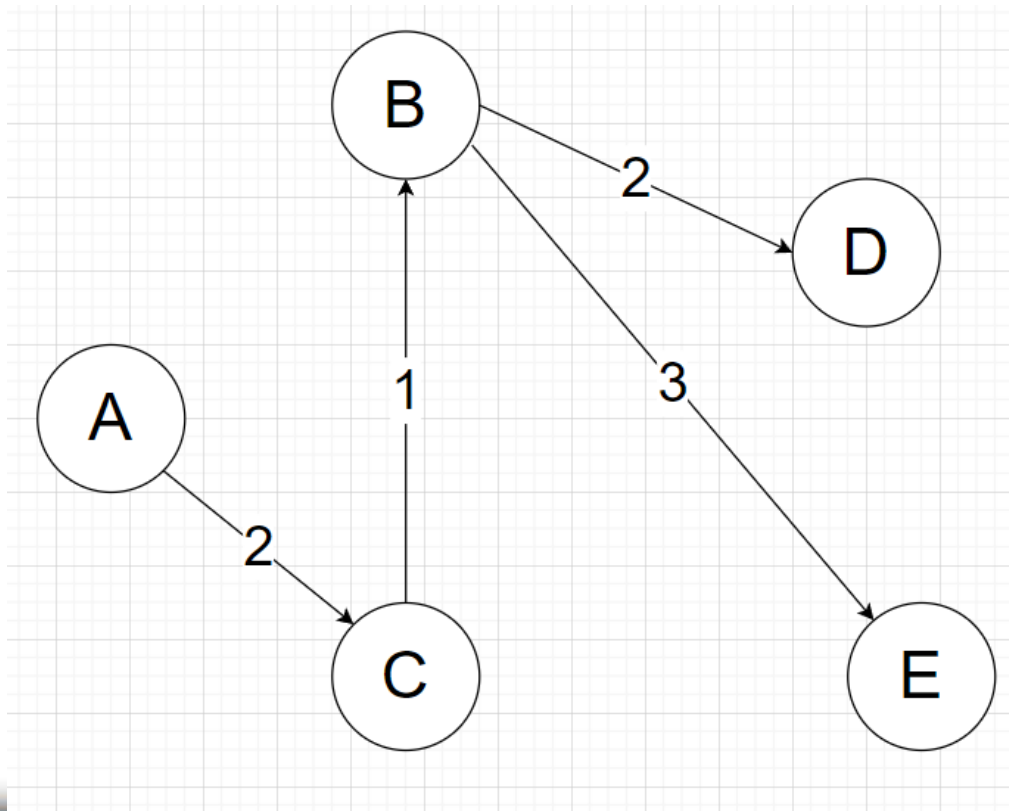
Unvisited nodes:

~~A~~, ~~B~~, ~~C~~, ~~D~~, ~~E~~



{ Dijkstra

- Vi har nu den korteste vej til alle de andre noder



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

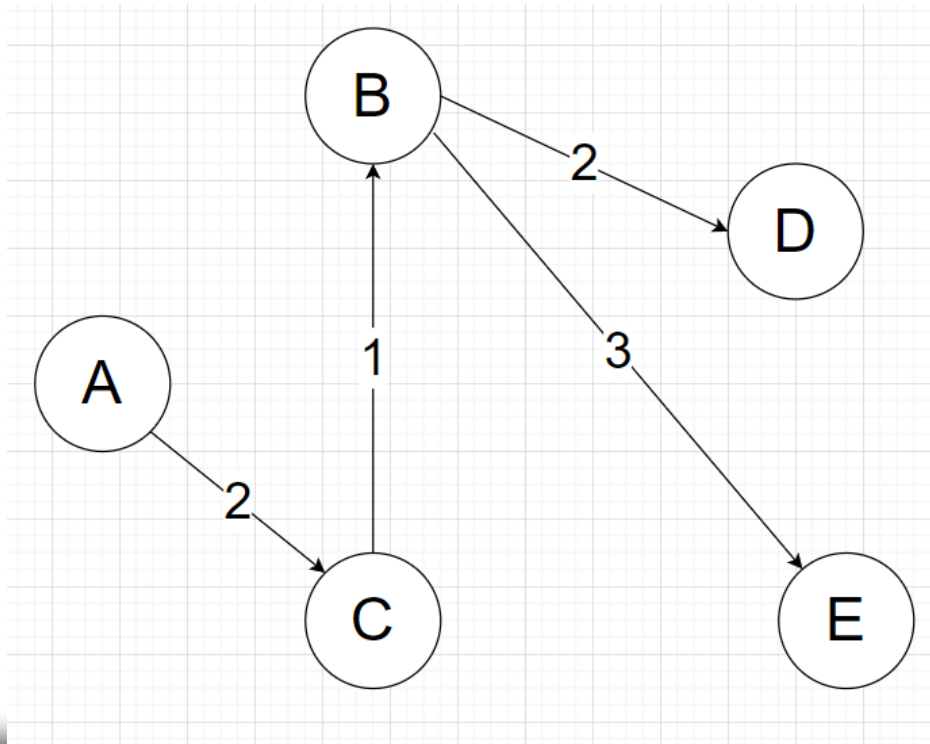
Unvisited nodes:

{~~A~~,~~B~~,~~C~~,~~D~~,~~E~~}



{ Dijkstra

- For at få selve path'en, kan vi backtracke fra den node vi gerne vil finde frem til.
- Selvfølgelig kunne vi også have gemt path'en undervejs sammen med vores distances. Dette tager dog mere memory.



Distances(fra A):

A:0

B:3

C:2

D:5

E:6

Unvisited nodes:

{~~A~~,~~B~~,~~C~~,~~D~~,~~E~~}



{ Dijkstra

<https://www.youtube.com/watch?v=pSqmA0-m7Lk&t=490s>

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each neighbor v of u still in Q:
14      alt ← dist[u] + Graph.Edges(u, v)
15      if alt < dist[v]:
16        dist[v] ← alt
17        prev[v] ← u
18
19   return dist[], prev[]
```

- Source = startnode
- V = node
- Q = unvisited
- Prev = parent/eller collection/opslag for tidligere.

Forklaring af Dijkstra:

Start med at sætte alle afstande til uendeligt, undtagen for kilden, som sættes til 0. Mens der stadig er noder at udforske: Vælg noden med den mindste kendte afstand. Opdater afstandene til dens naboer, hvis der findes en kortere vej. Returner de beregnede afstande fra kilden til alle noderne.



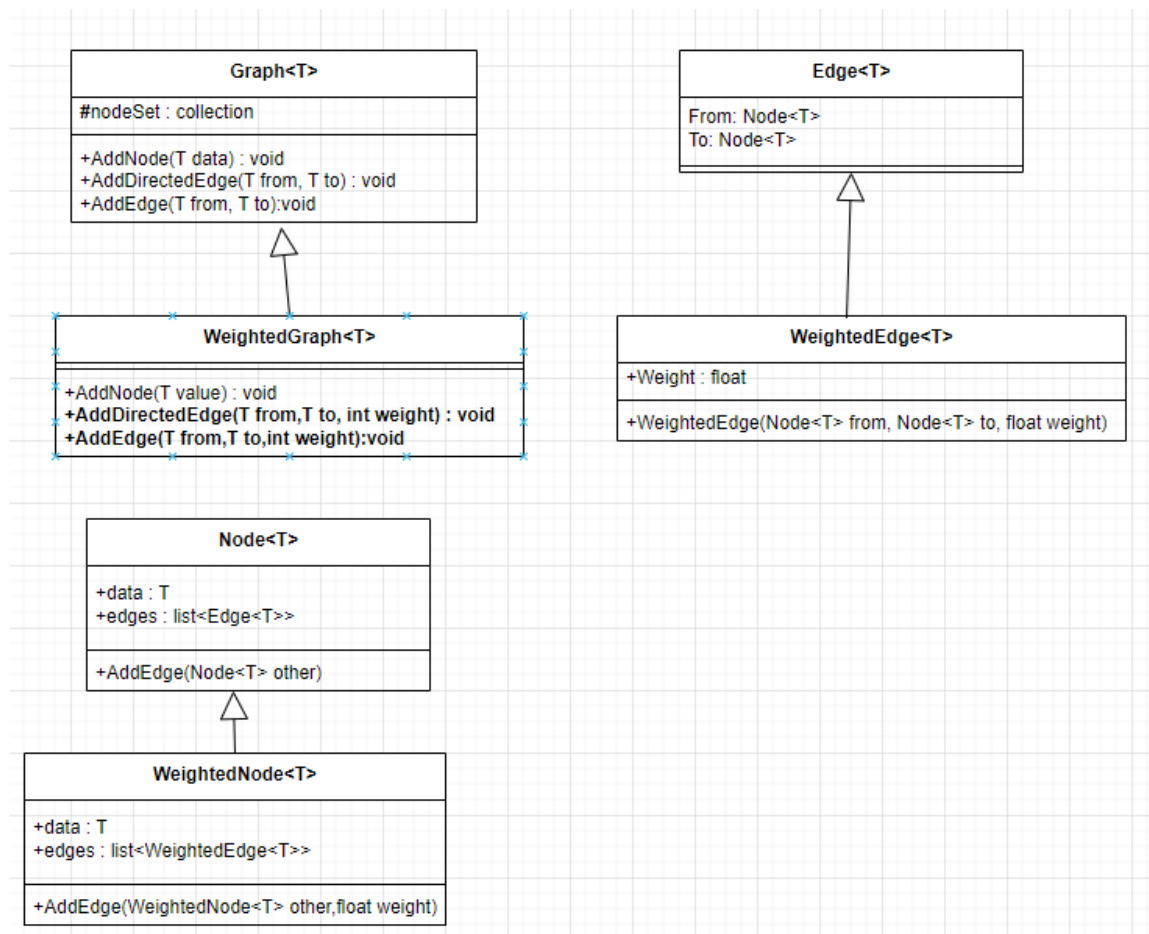
{ Dijkstra

- $O(V^2)$ (V er antallet af noder i grafen) man bruger V til forklaring, da det handler om en graf
- Mere om algoritmen [her](#)
 - Wiki link, kompleksitet, bevis for korrekthed.
- Dijkstras algoritme virker kun på vægtede grafter med udelukkende positive edges.
- Dvs før vi kan bruge den skal vi have udvidet vores graph library.
- Vi skal have vægt på kanterne.
- Der er to måder vi kan gribe dette an på



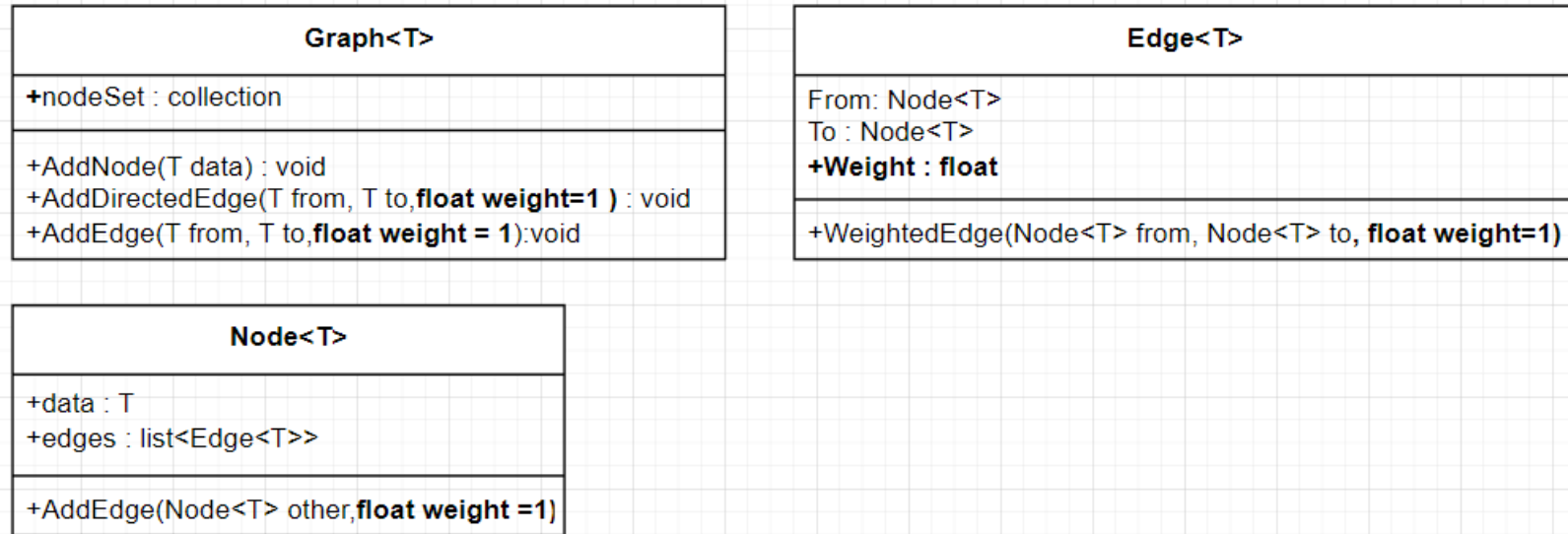
Dijkstra

- Vi kan oprette en specialiseret version af graph-klassen ved hjælp af polymorfi



{ Dijkstra

- Udvidelse af grundklasse, med default værdier:



- Mindre arbejde, bagud kompatibel, men mere memory...
- Nok stadig den jeg selv ville foretrække... 😊



{ Øvelse

- Udvid jeres grafer til at understøtte vægtede edges
- Implementer en løsning på Dijkstras algoritme
- Brug blot grafen fra slides, hvis I har brug for facit 😊
- Udskriv det færdigt opdaterede distances map
- Bonus:
 - Udskriv path'en fra **from** til **to** via inputtet.
 - Tjek wiki linket for inspiration: [her](#)



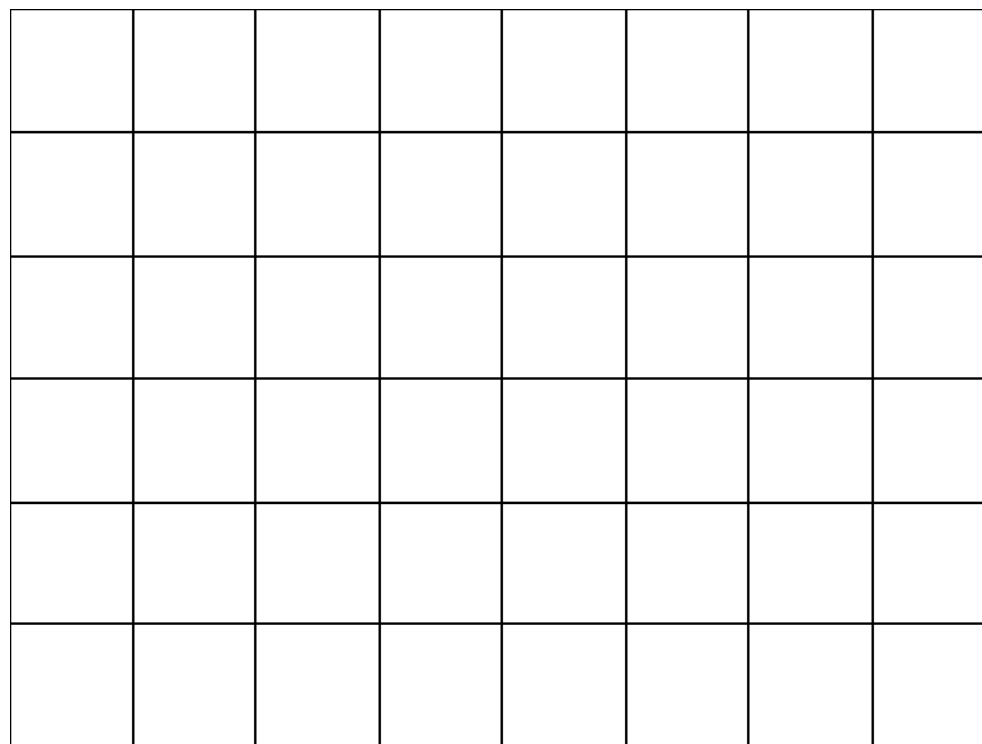
A* Algoritmen

forklare den på ca samme måde som eksemplet
<https://www.youtube.com/watch?v=aIU04hvz6L4>



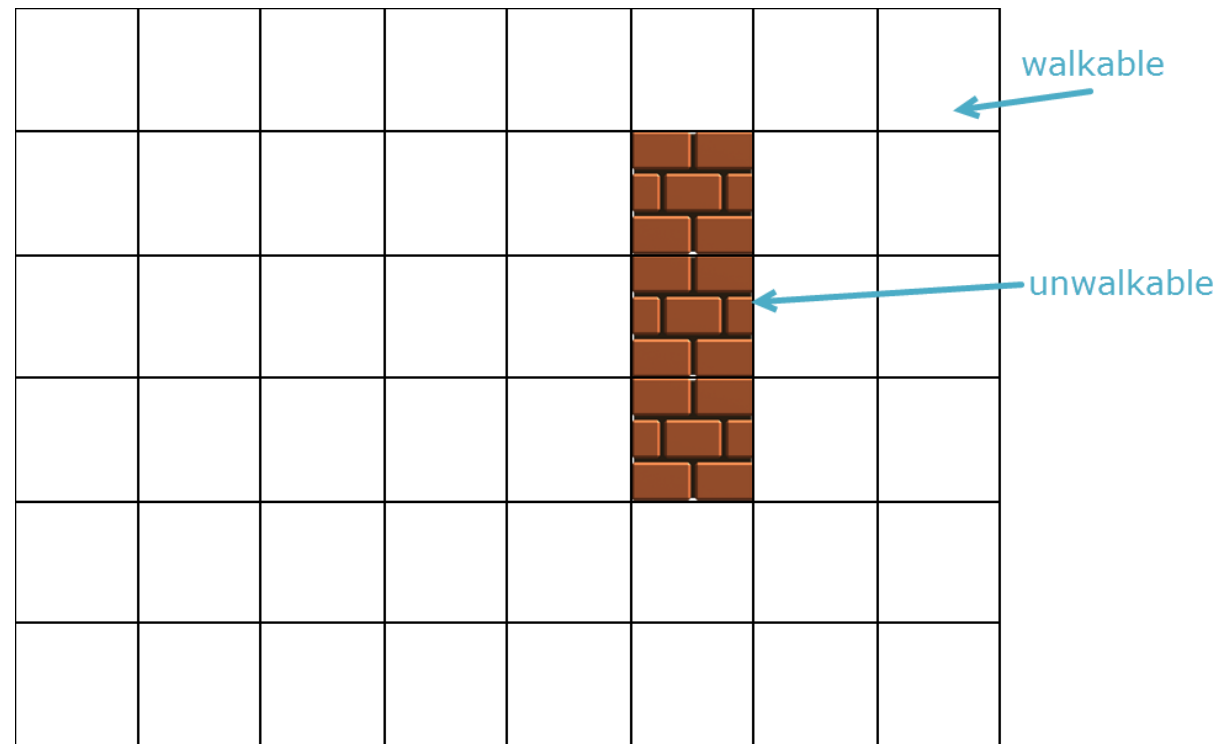
A* algoritmen

- Algoritmens søgeområde inddeles i et grid med noder



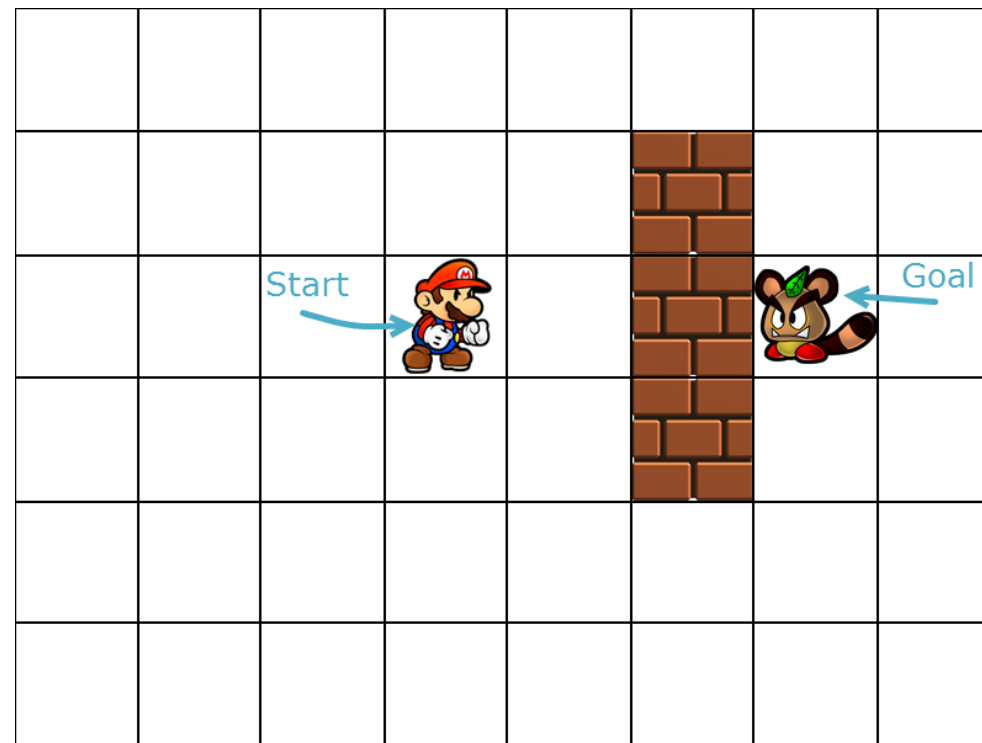
A* algoritmen

- Disse noder kan enten være walkable eller unwalkable



A* algoritmen

- En start node og en slut node vælges i søgeområdet



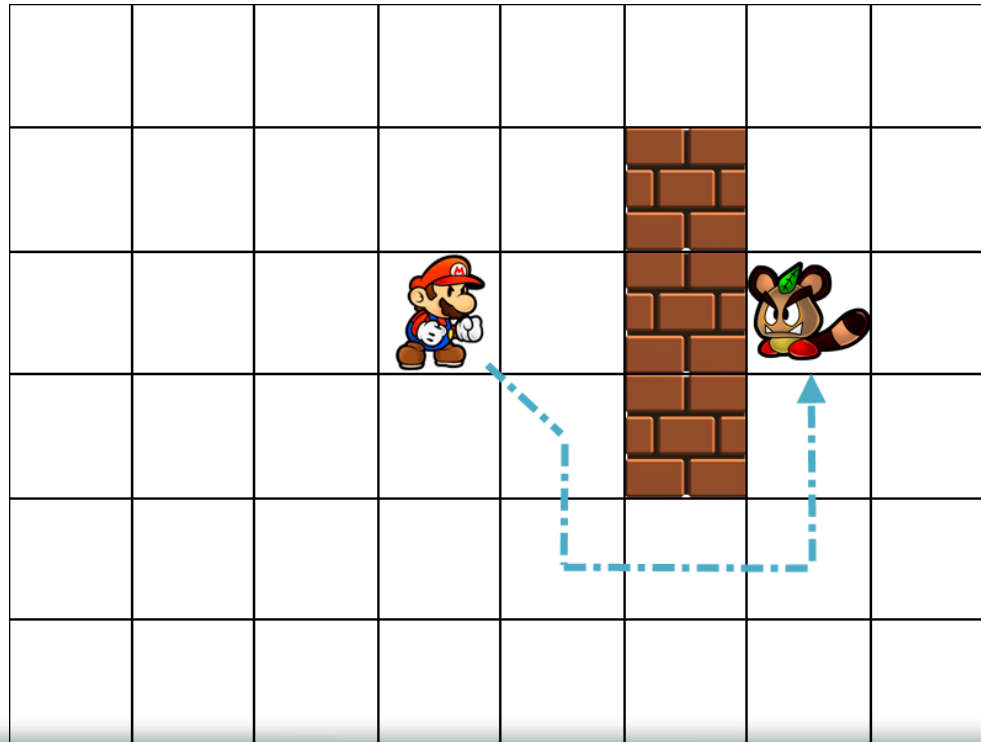
A* algoritmen

- Algoritmen arbejder ud fra to lister
 - Åben liste (open list)
 - En liste over noder, som endnu ikke er kandidater til at blive en del af stien
 - Man kan se på open listen, som en liste af noder, der skal undersøges
 - Lukket liste (closed list)
 - Består af bekræftede kandidater til ruten, dvs. at noderne **kan** blive en del af den endelige sti
 - Disse noder er blevet undersøgt



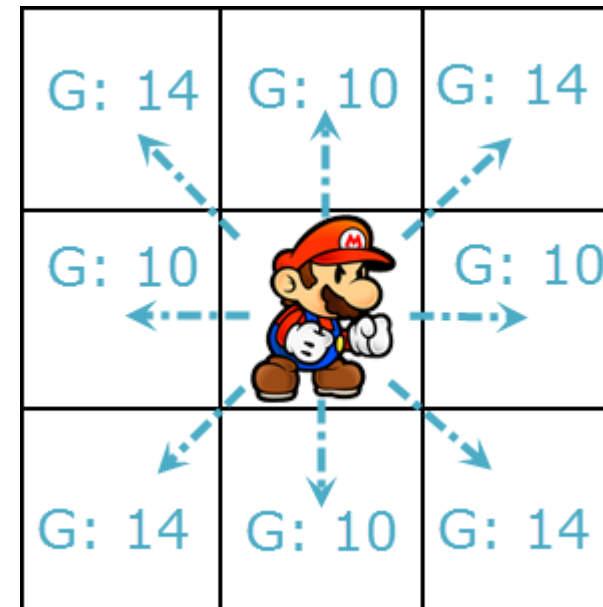
{ A* algoritmen

- Algoritmen søger gennem noderne for at finde den korteste sti fra node A til node B
 - Funktionen $f(x) = g(x) + h(x)$ bruges til at udvælge noder til ruten



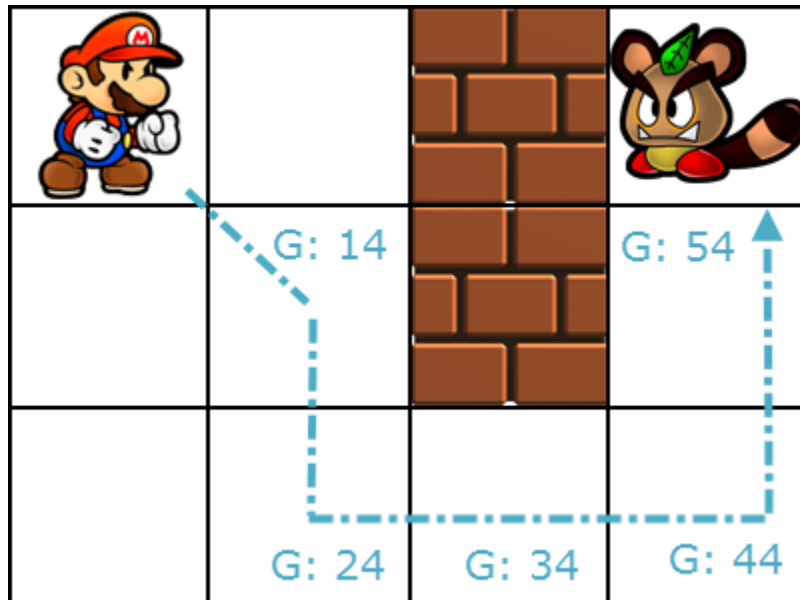
{ A* algoritmen

- $g(x)$ = Hvad koster det at bevæge sig fra start til noden gennem den genererede sti
 - Det koster 10 at flytte sig horisontalt eller vertikalt
 - Det koster 14 af flytte sig diagonalt (kommer fra den pythagoræiske læresætning)
 - Egentlig $\sqrt{200} \approx 14.142$
 - 14 fylder mindre i slides.



{ A* algoritmen

- **G** Beregnes ved at tage **g** prisen fra parent noden og derefter tilføje 10 eller 14 alt efter om der blev flyttet diagonalt eller ortogonalt for at komme til den nuværende node



{ A* algoritmen

- $h(x)$ = den estimerede pris for at bevæge sig fra en node til målet
 - Bliver ofte refereret til som heuristik
 - Det er et gæt på hvad det koster at flytte fra noden til målet
 - Der tages ikke højde for forhindringer (unwalkable noder) og diagonale bevægelser
 - Antallet af felter mellem de to noder optælles og ganges med 10

H: 50	H: 40	H: 30		H: 10
H: 40		H: 20		
H: 50	H: 40	H: 30		H: 10

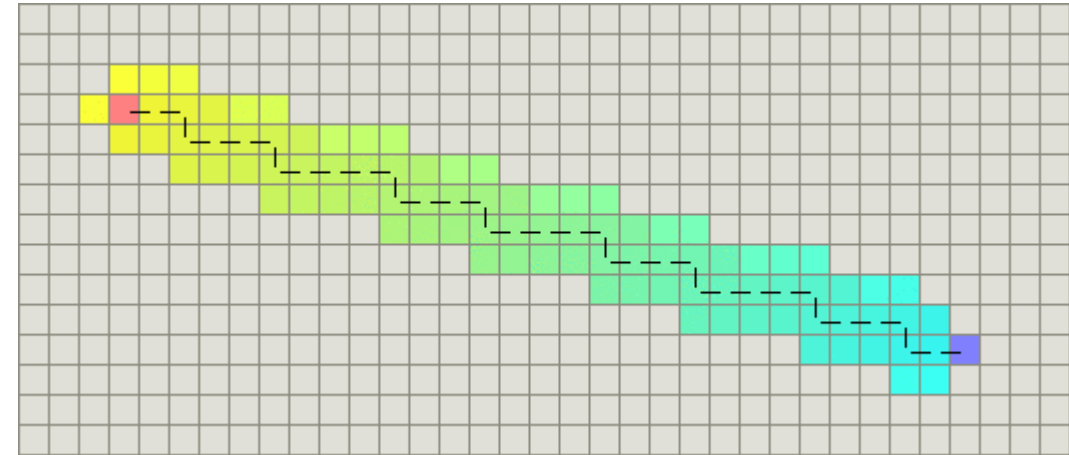


A* algoritmen

- **H** Bliver ofte udregnet som direkte linjer mellem noden og målet - dette kaldes "Manhattan princippet"
 - Denne værdi må ikke overestimeres. Hvis den overestimeres findes den korteste rute ikke
 - Jo mere præcis estimeringen er - jo hurtigere er algoritmen
 - Teknisk set kan denne metode godt overestimere resterende distance, men den er let at forstå og arbejde med
 - Den giver næsten altid den korteste sti. Hvis det ikke er tilfældet ligger den meget tæt på
 - Metoden har også den fordel, at den er meget hurtig i forhold til mange alternativer
 - Se mere her:
 - <http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>



- Eksempel på udregning af $H(x)$ værdi
 - (Tager ikke højde for diagonal bevægelse)



```
public class HeuristicCalculator
{
    0 references
    public static float CalculateDistanceToGoal(Node currentNode, Node goalNode, float distanceMultiplier)
    {
        int horizontalDistanceToGoal = Math.Abs(currentNode.x - goalNode.x);
        int verticalDistanceToGoal = Math.Abs(currentNode.y - goalNode.y);
        return distanceMultiplier * (horizontalDistanceToGoal + verticalDistanceToGoal);
    }
}
```



A* algoritmen

- $f(x)$ beregnes ved $g(x) + h(x)$

			
	G: 14 H: 30 F: 44		G: 54 H: 10 F: 64
	G: 24 H: 40 F: 64	G: 34 H: 30 F: 64	G: 44 H: 20 F: 64



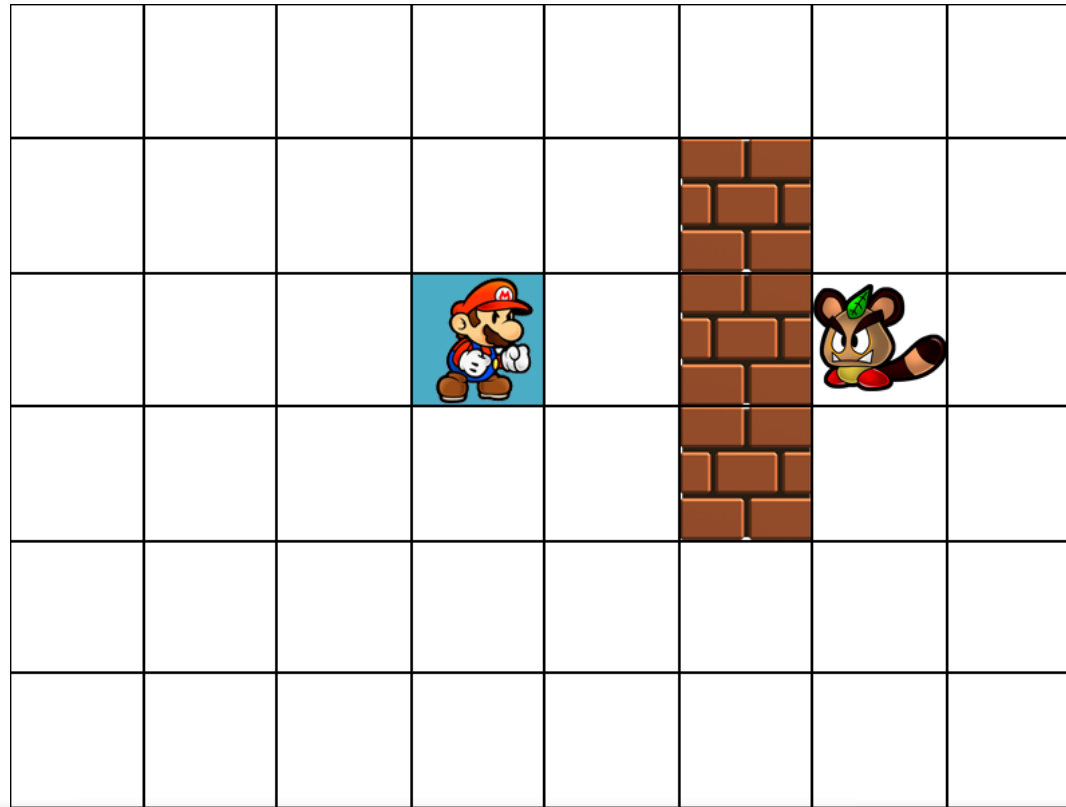


Eksempel



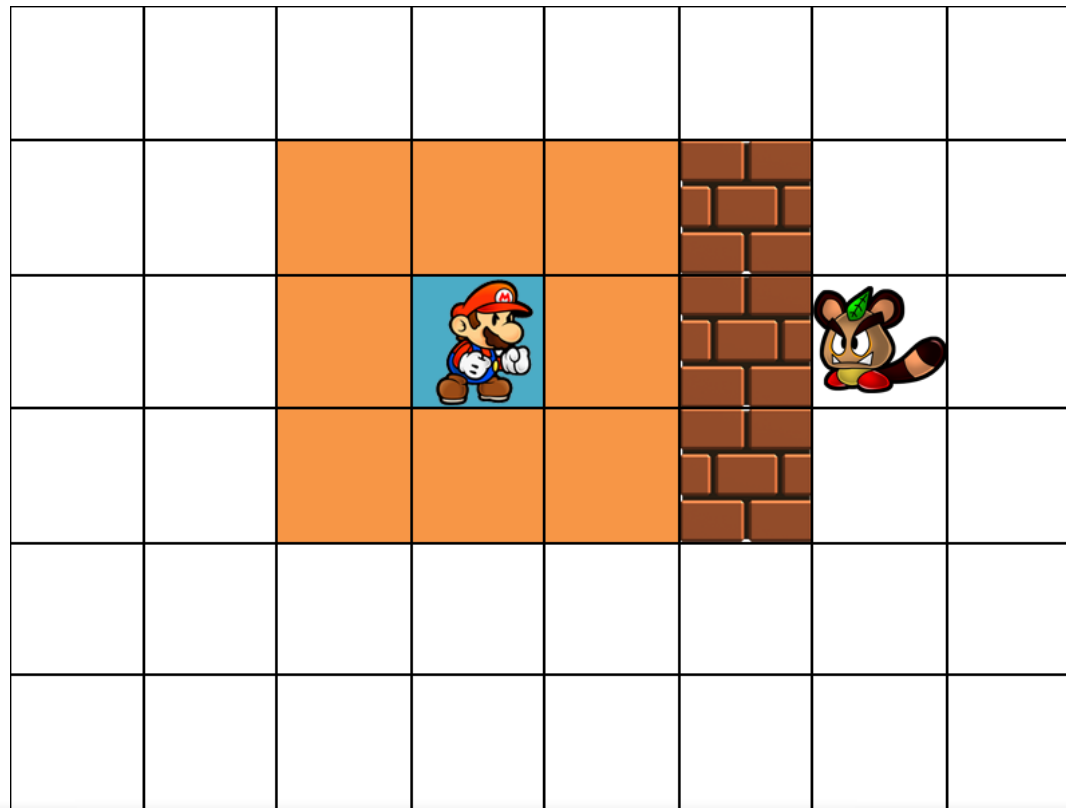
{ Eksempel

1. Tilføj start node til den åbne liste



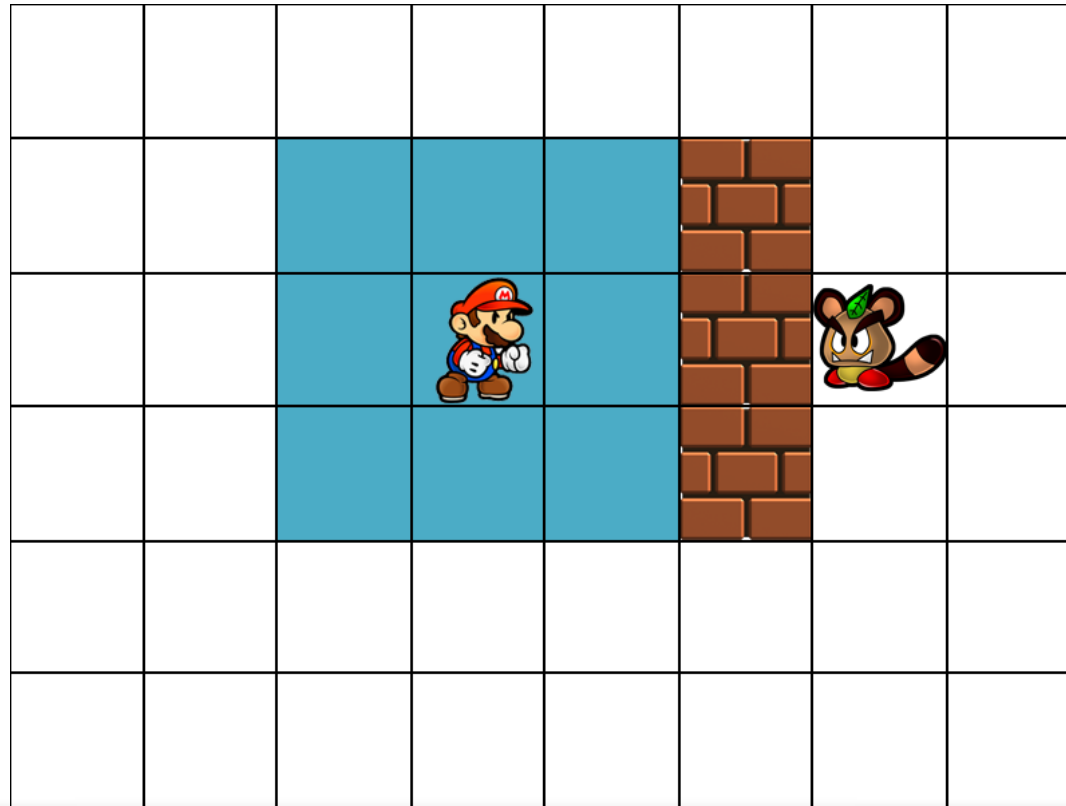
{ Eksempel

2. Se på nodernes naboer, unwalkable noder ignoreres



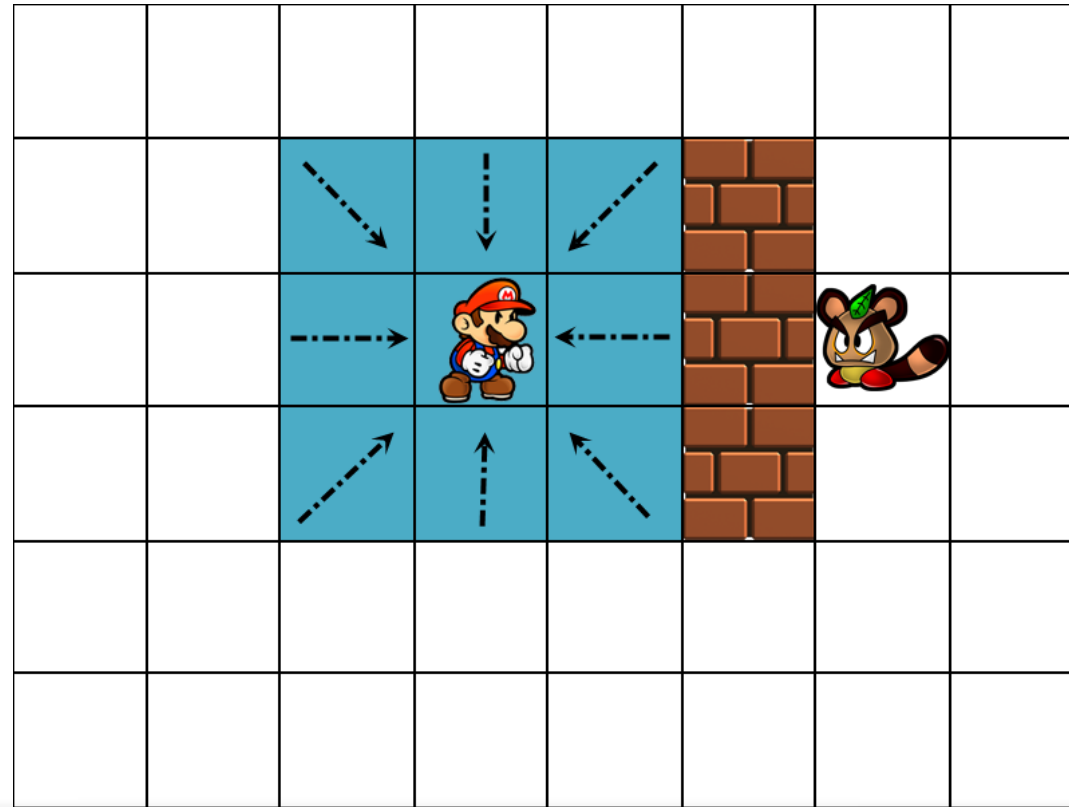
{ Eksempel

3. Naboerne tilføjes til den åbne liste



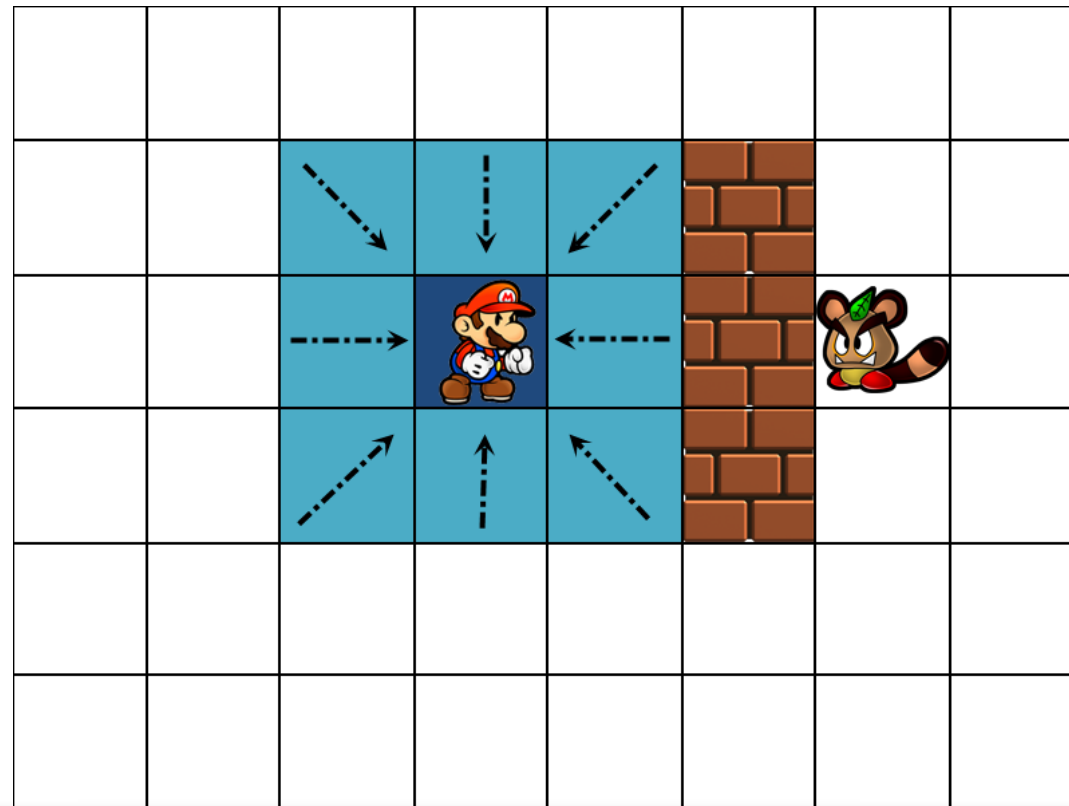
{ Eksempel

4. Start noden noteres som parent til nabonoderne



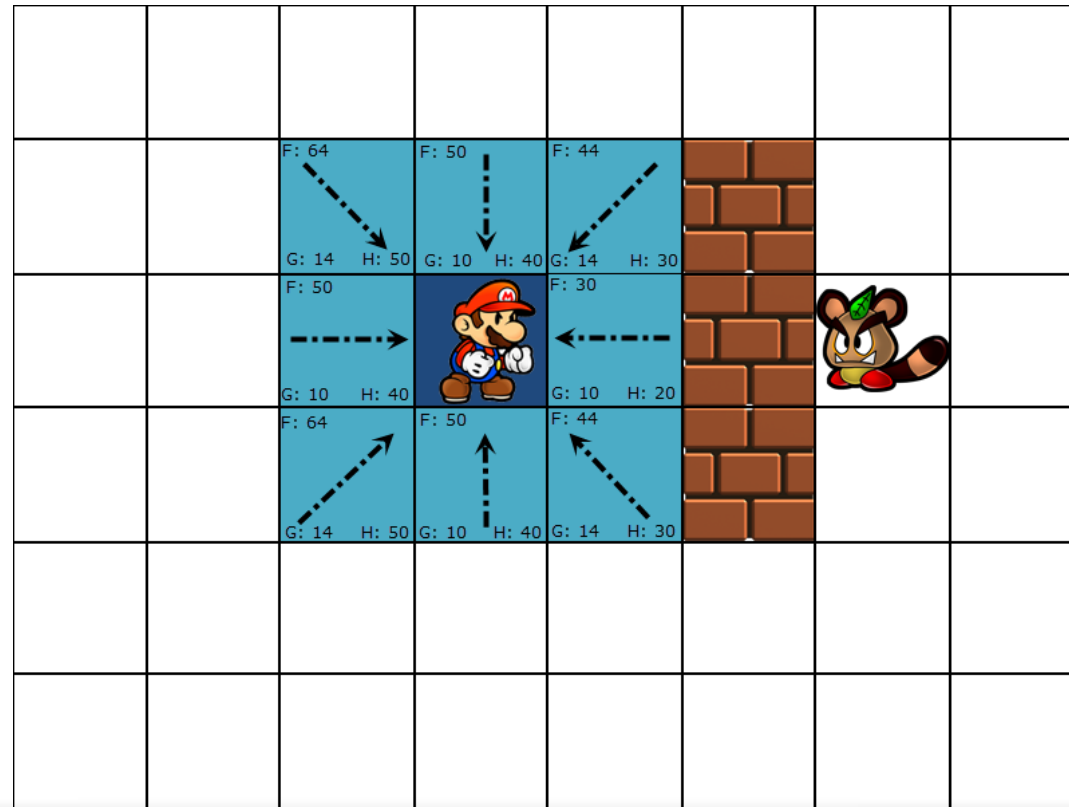
{ Eksempel

5. Start noden flyttes fra den åbne til den lukkede liste



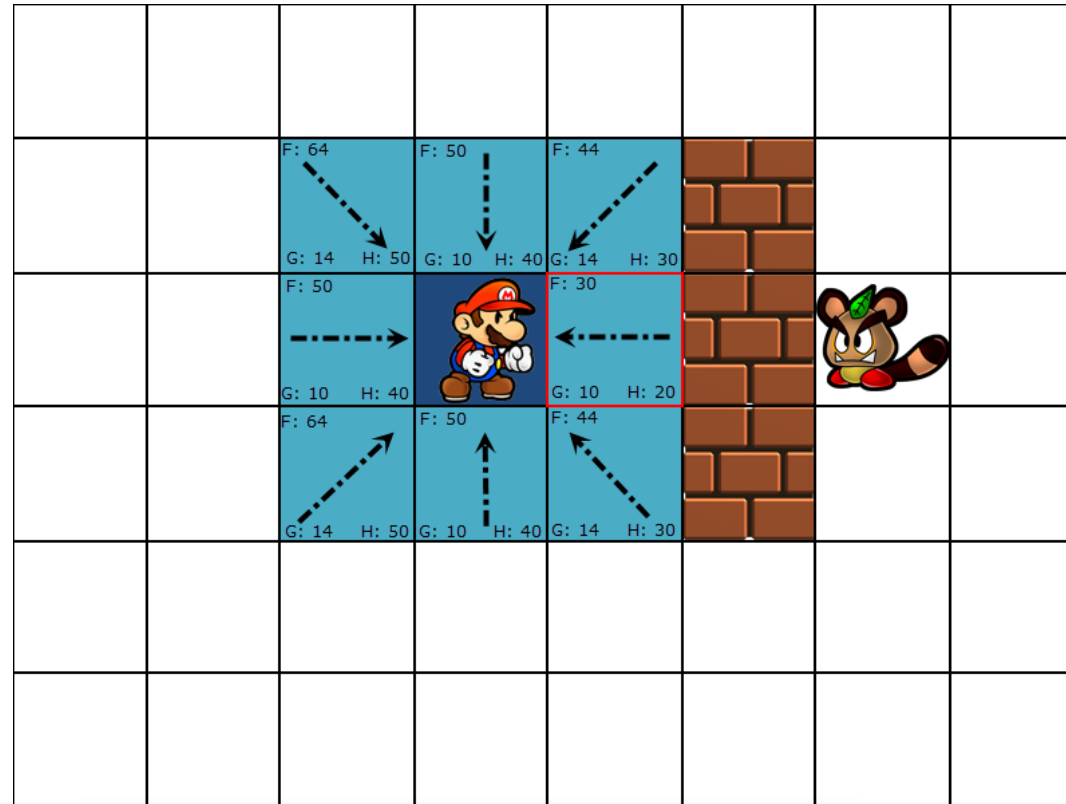
{ Eksempel

6. nodernes score beregnes som beskrevet tidligere



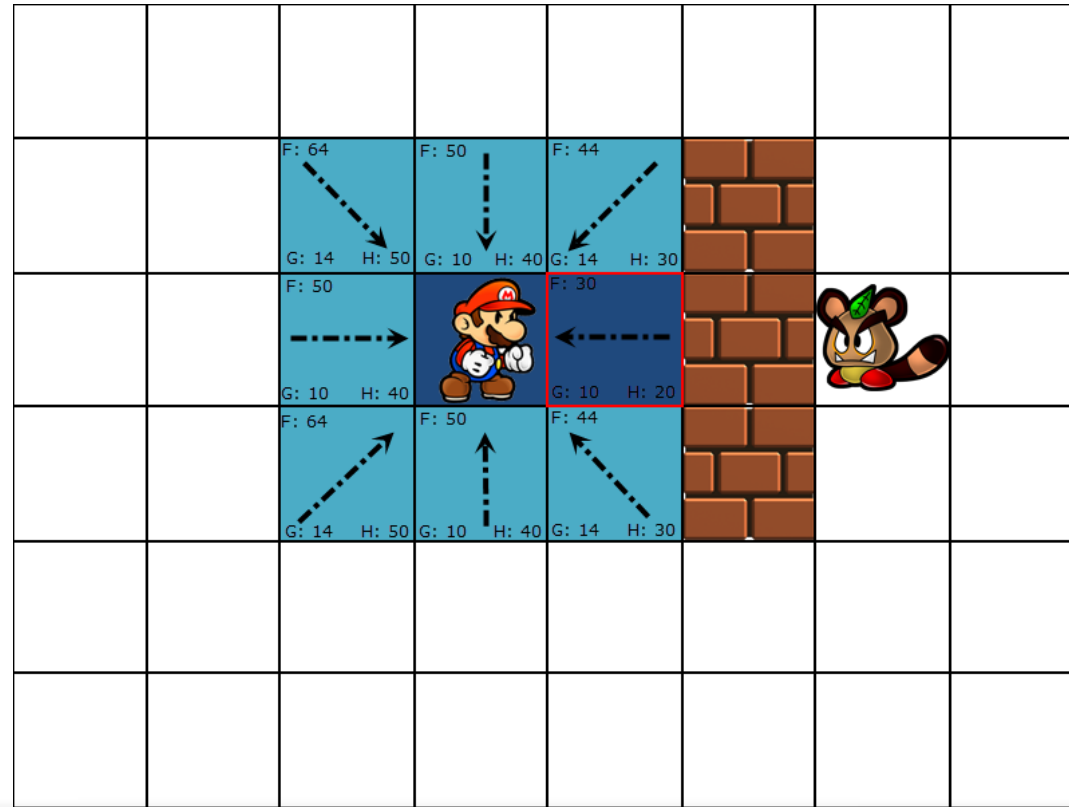
Eksempel

7. Søgningen fortsættes ved at vælge den node fra den åbne liste med den laveste F score



{ Eksempel

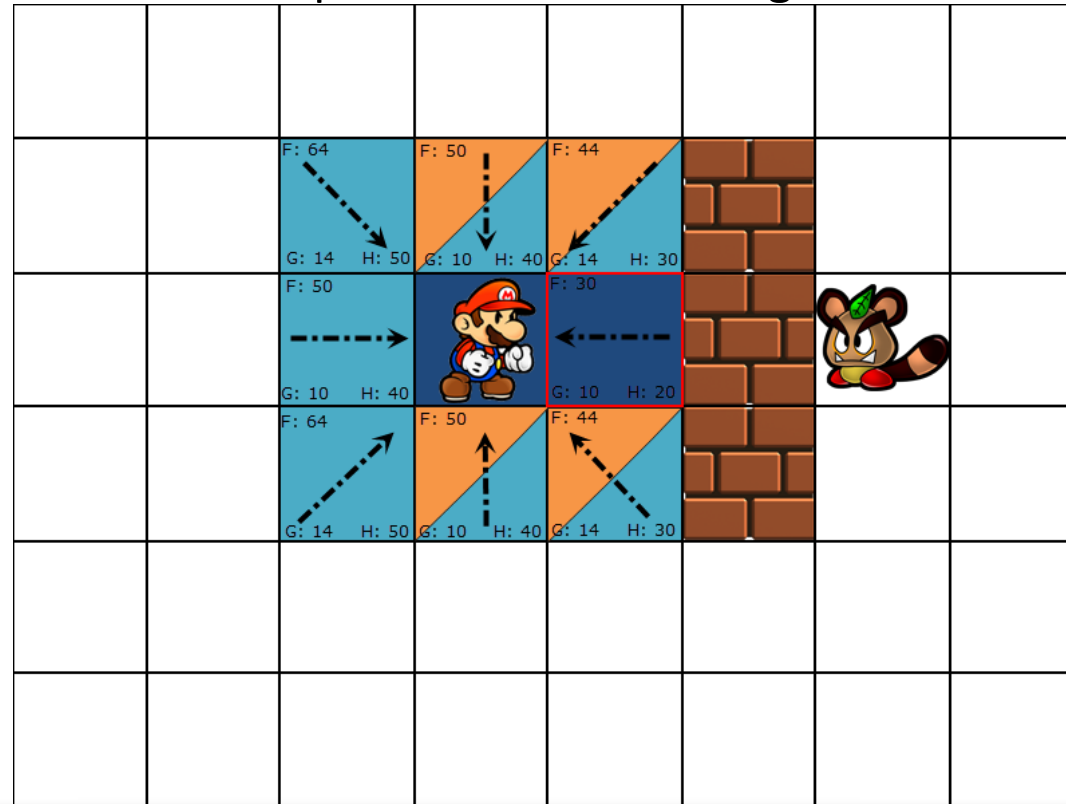
8. Noden flyttes fra den åbne liste til den lukkede liste



{ Eksempel

9. Undersøg alle naboer

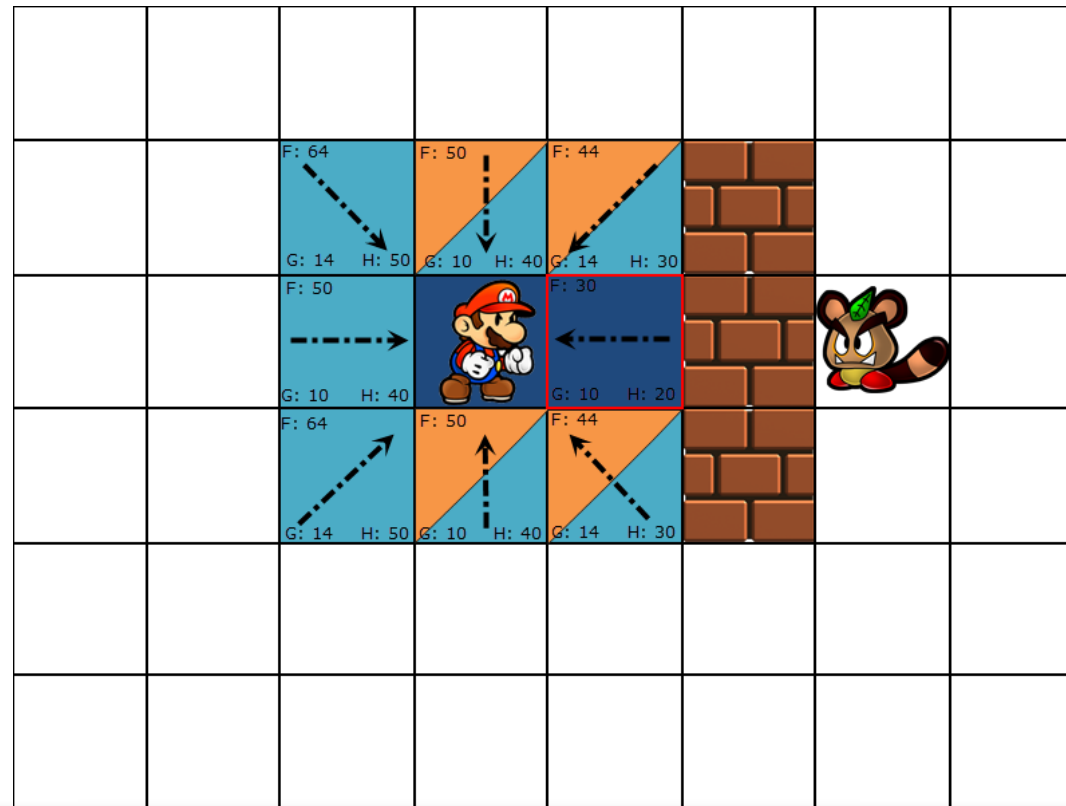
1. Unwalkable noder samt noder på den lukkede liste ignoreres



{ Eksempel

9. Undersøg alle naboer

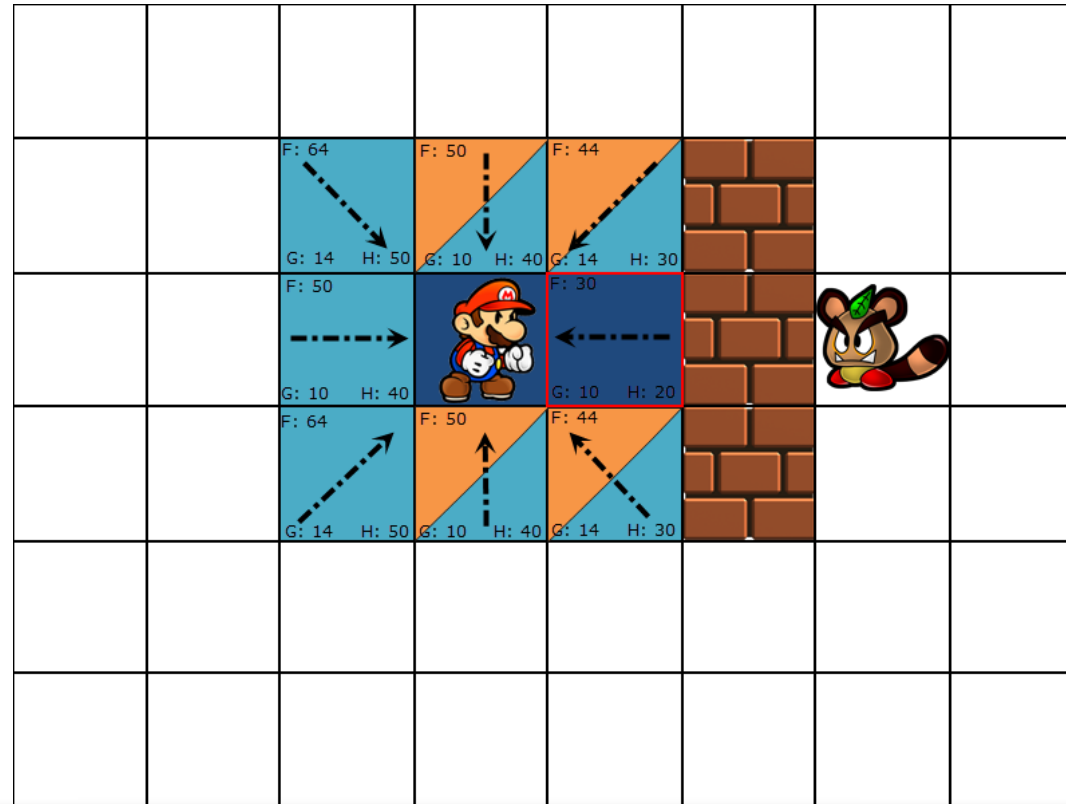
2. Naboer, som ikke allerede er på den åbne liste tilføjes til den (alle naboer er allerede på listen)



{ Eksempel

9. Undersøg alle naboer

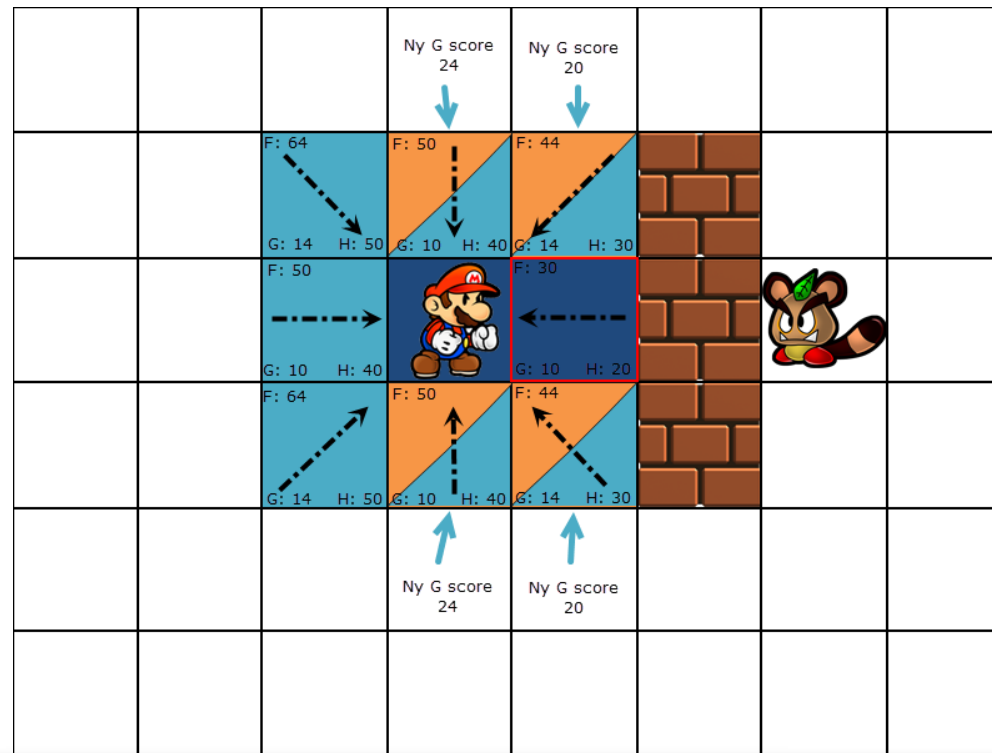
3. Den nuværende node gøres til parent til de nye noder på den åbne liste, nodernes G, H og F score beregnes



Eksempel

9. Undersøg alle naboer

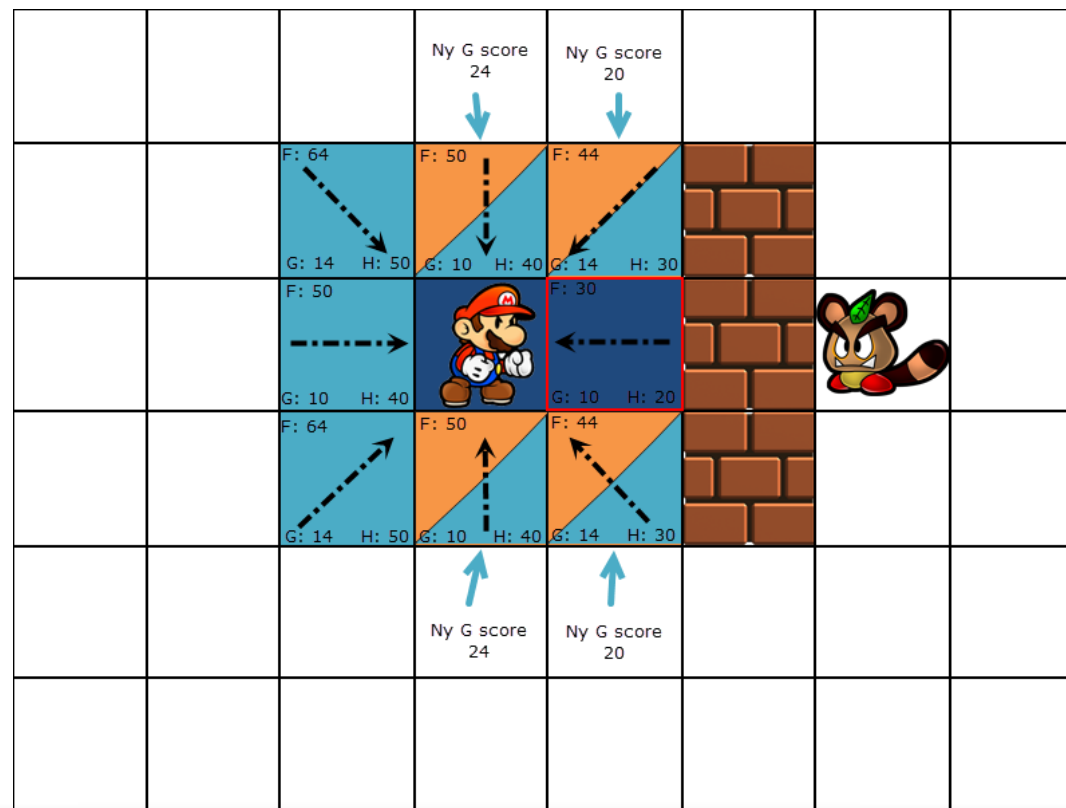
4. Hvis naboer allerede er på den åbne liste, kontrolleres det om stien gennem den nuværende node er et bedre alternativ. Dvs. er G scoren lavere hvis vi går igennem den nuværende node



Eksempel

9. Undersøg alle naboer

5. Hvis naboens G scoren er lavere, ændres dens parent til den **nuværende node**, herefter genberegnes dens F og G scores. Hvis scoren ikke er lavere gør vi intet. I dette tilfælde er scoren ikke lavere



Eksempel

10. Søgningen fortsætter ved at gentage fra punkt 7 indtil:

- 1) Den åbne liste er tom
 - Dvs. at stien ikke blev fundet
- 2) Målet er tilføjet til den lukkede liste
 - Når målet er tilføjet, kan stien findes ved at backtracke gennem nodernes parents



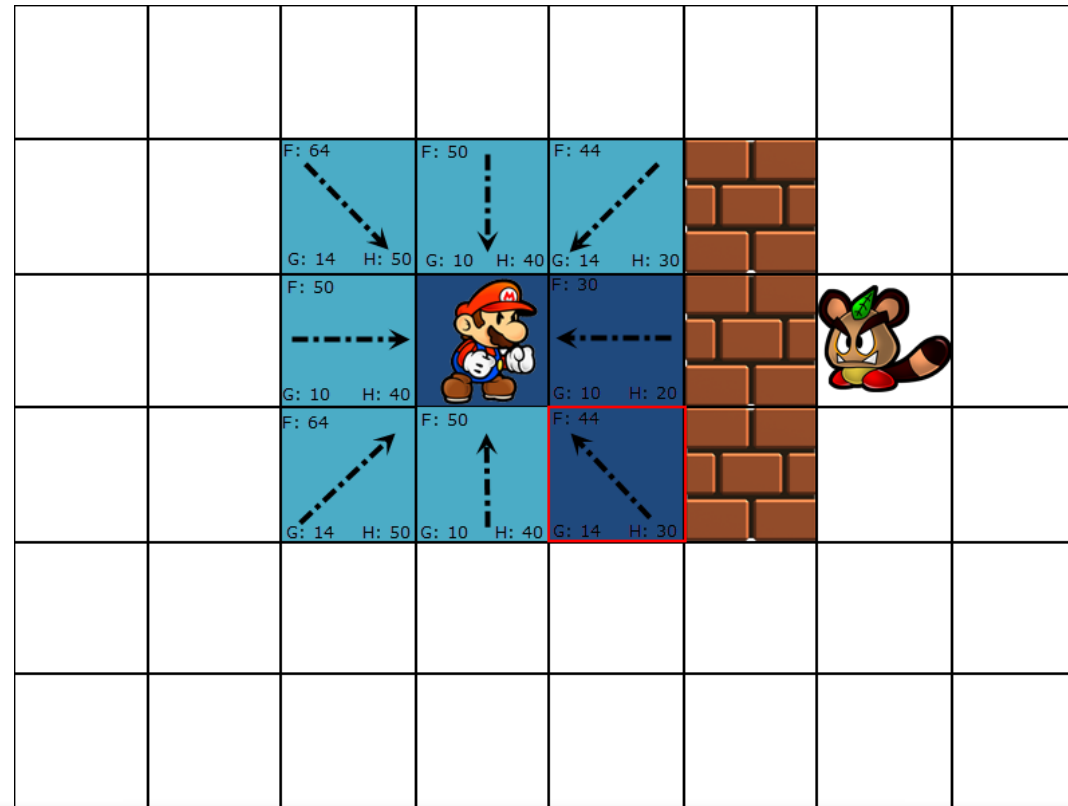
Eksempel

- NOTE: I følgende eksempel ignoreres beregninger med noder, som ikke har noget at gøre med den endelige sti. Disse er udeladt for at simplificere eksemplet
- Dvs. at den korrekte Astar algoritme ville bruge flere træk for at finde den endelige sti
- Et endeligt billede af den korrekte sluttilstand kan ses efter eksemplet



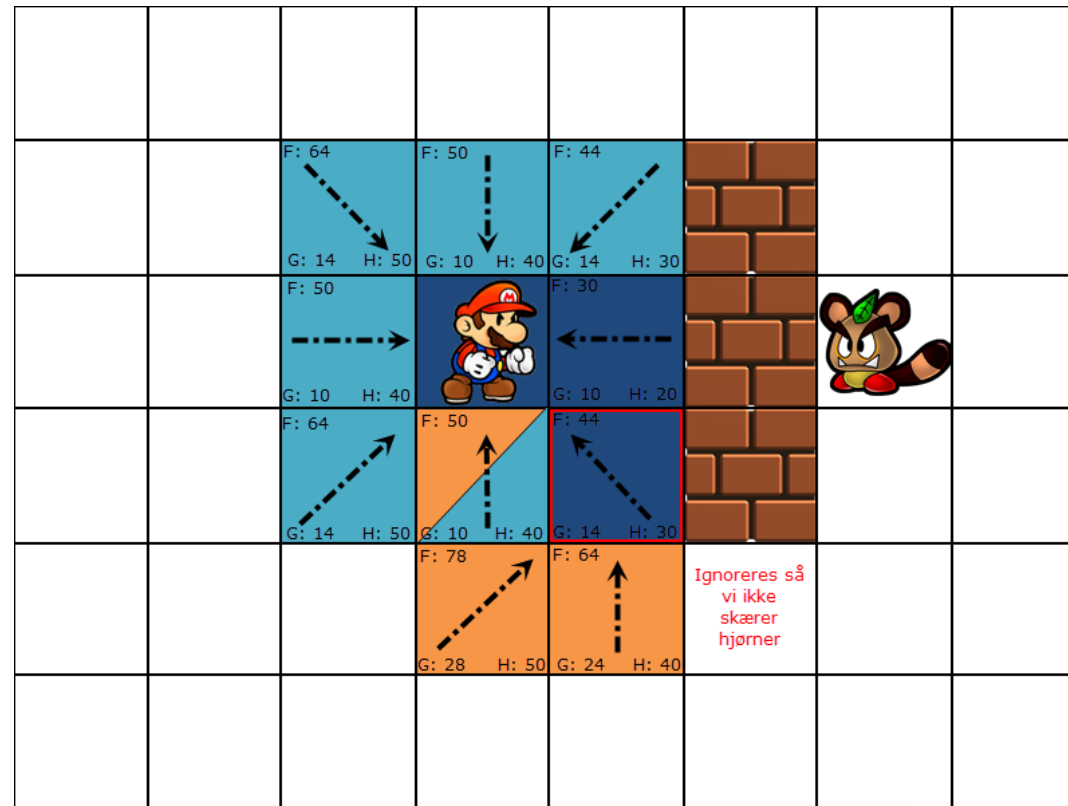
Eksempel

- noden med den laveste F score på den åbne liste vælges og flyttes til den lukkede liste



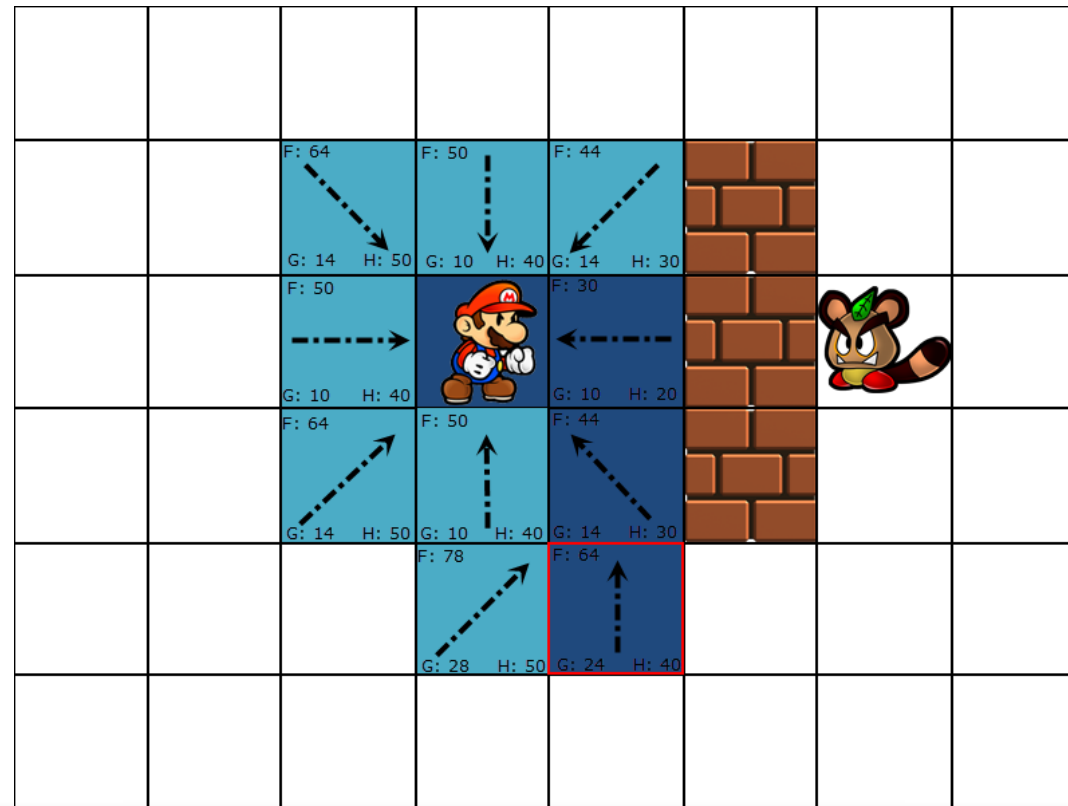
Eksempel

- Naboer undersøges
 - Parents og G,H,F værdier sættes, alternativ G score beregnes på naboer, som allerede var på listen



{ Eksempel

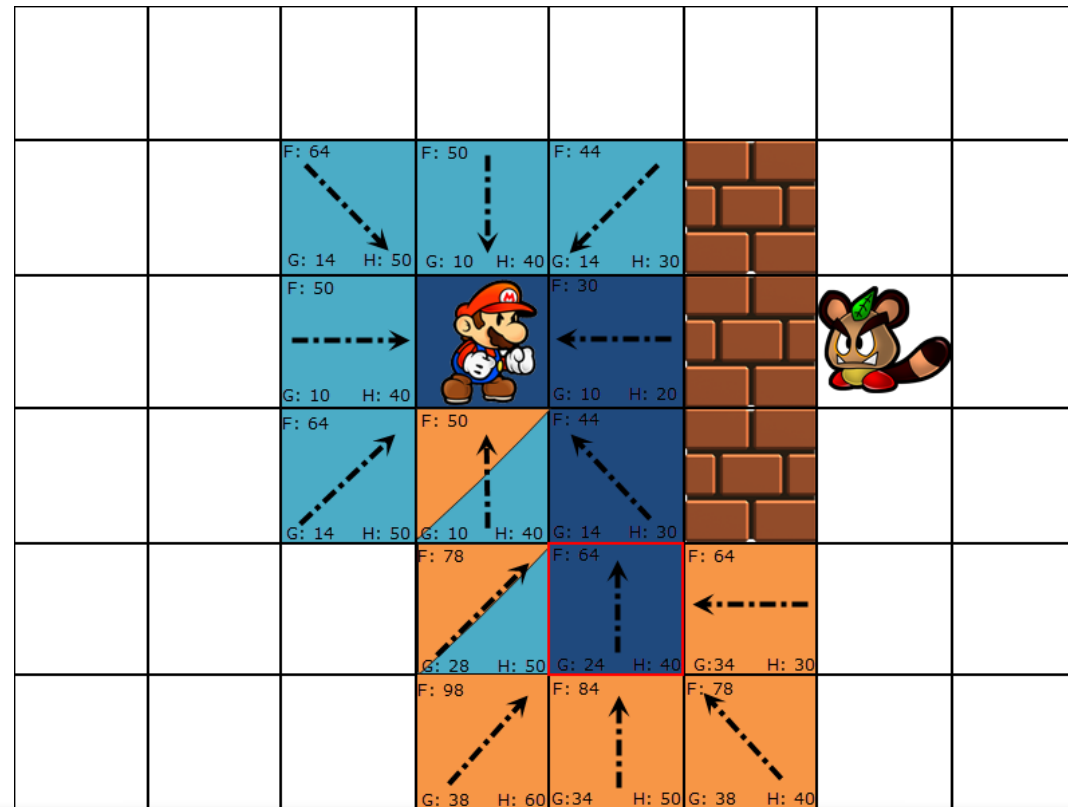
- noden med laveste F værdi vælges og flyttes fra den åbne til den lukkede liste



Eksempel

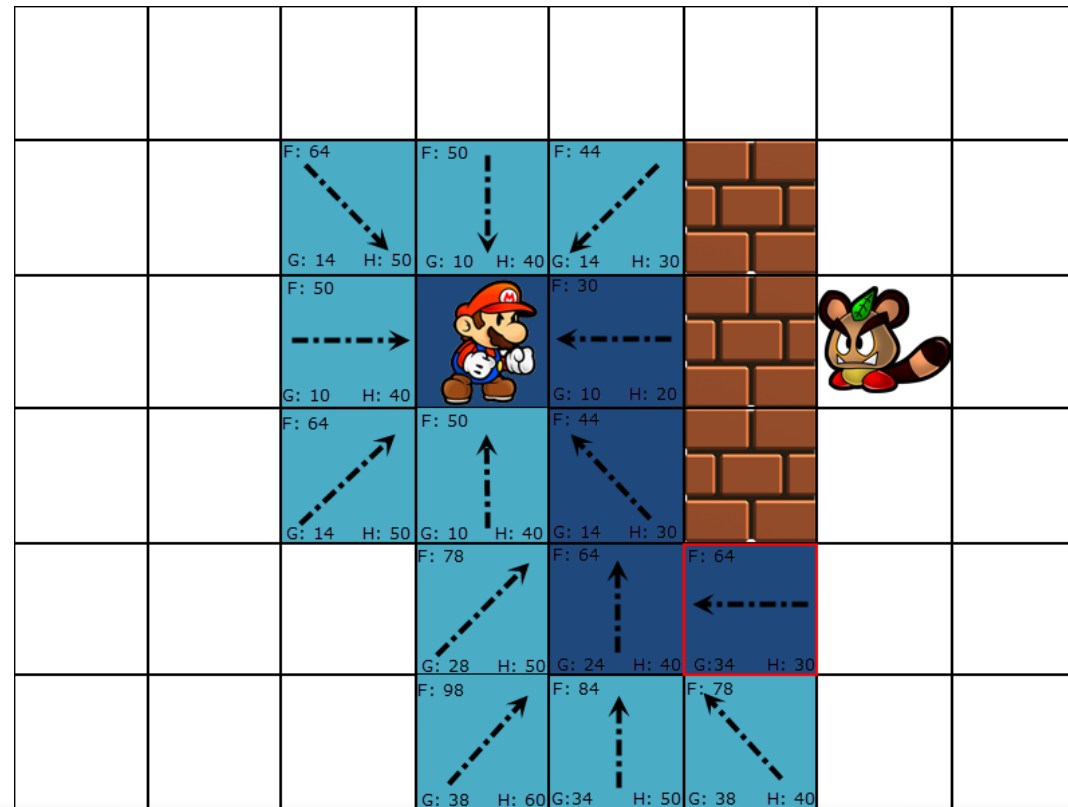
- Naboer undersøges

- Parents og G,H,F værdier sættes, alternativ G score beregnes på naboer, som allerede var på listen



{ Eksempel

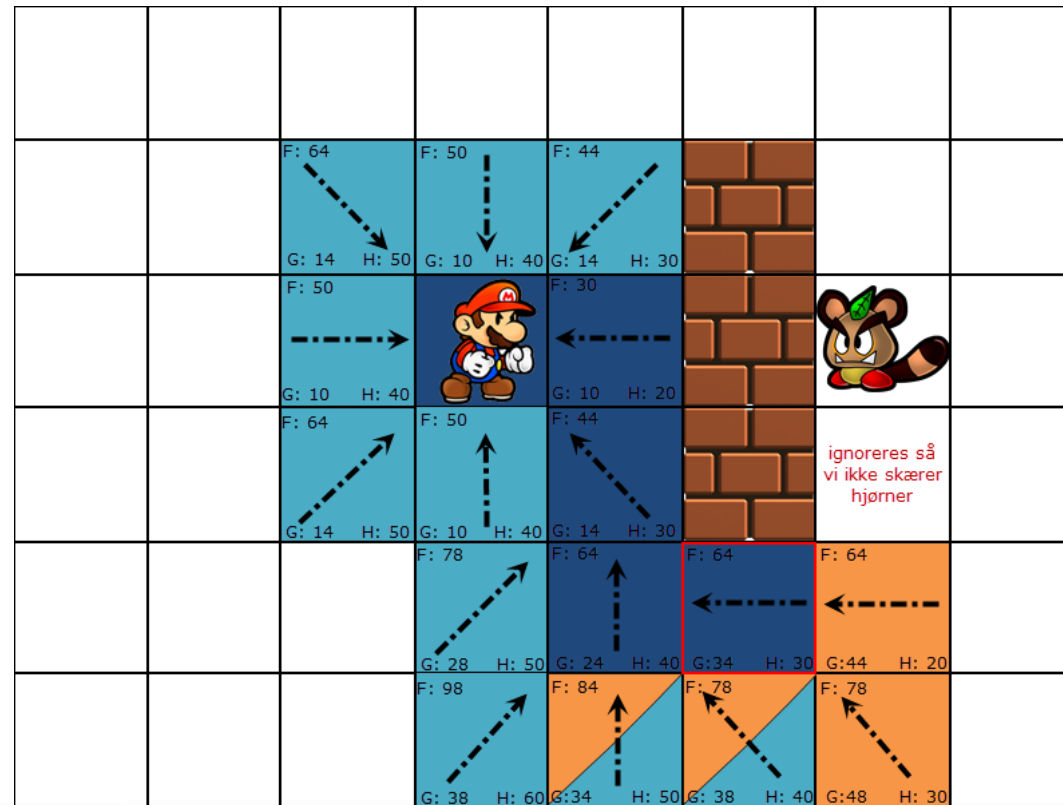
- noden med laveste F værdi vælges og flyttes fra den åbne til den lukkede liste



Eksempel

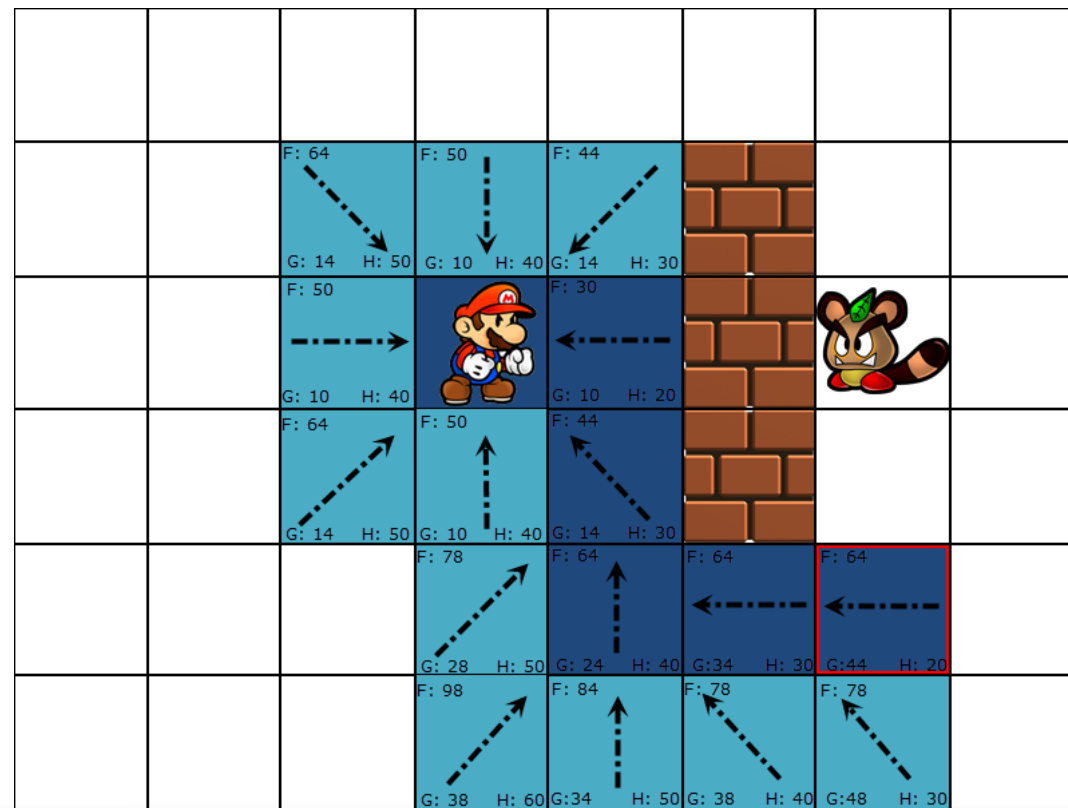
- Naboer undersøges

- Parents og G,H,F værdier sættes, alternativ G score beregnes på naboer, som allerede var på listen



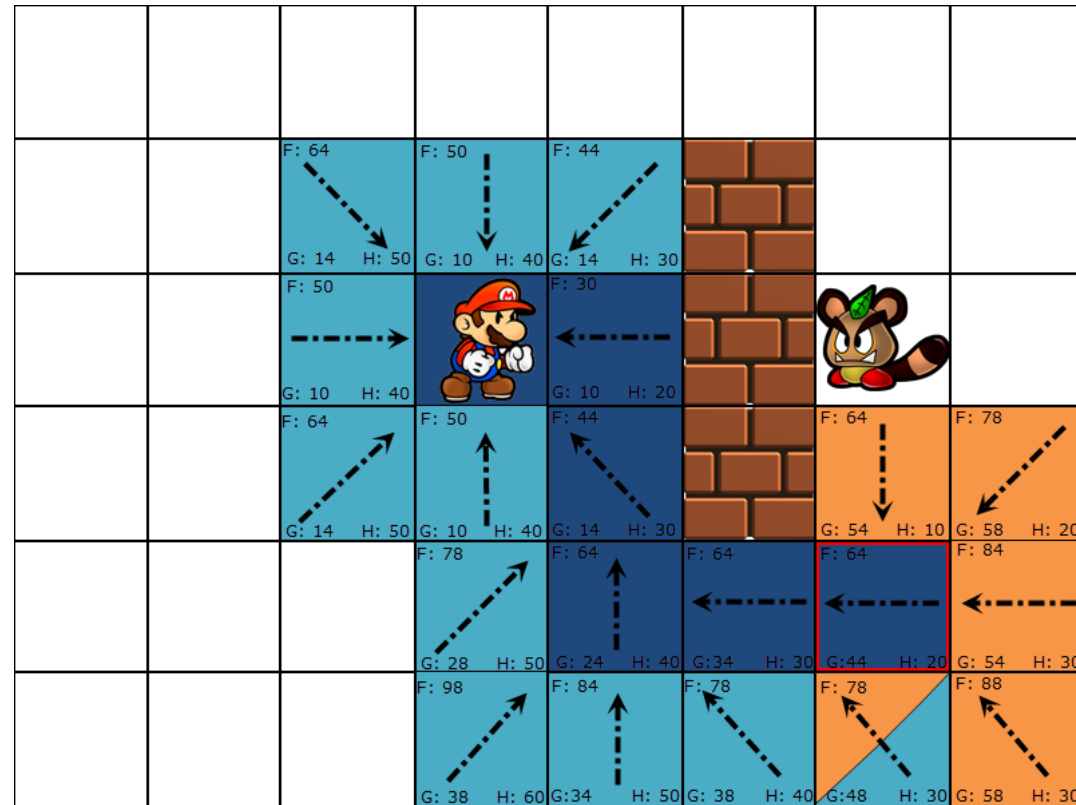
Eksempel

- noden med laveste F værdi vælges og flyttes fra den åbne til den lukkede liste



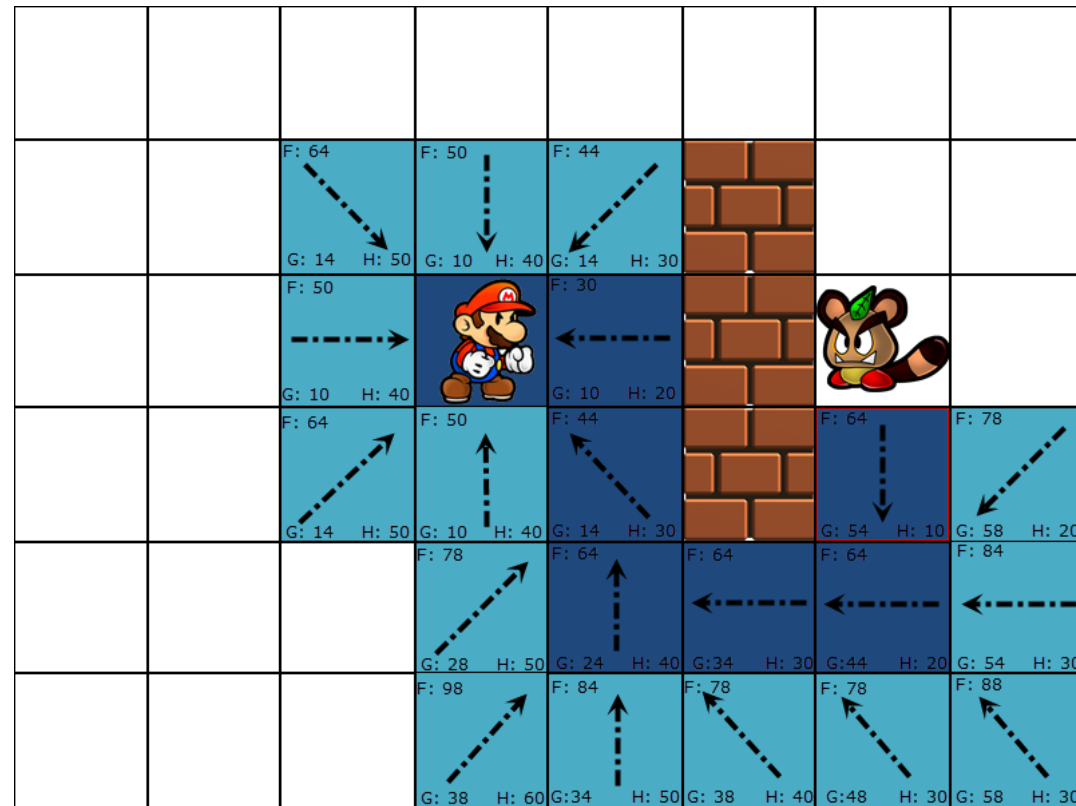
Eksempel

- Naboer undersøges
 - Parents og G,H,F værdier sættes, alternativ G score beregnes på naboer, som allerede var på listen



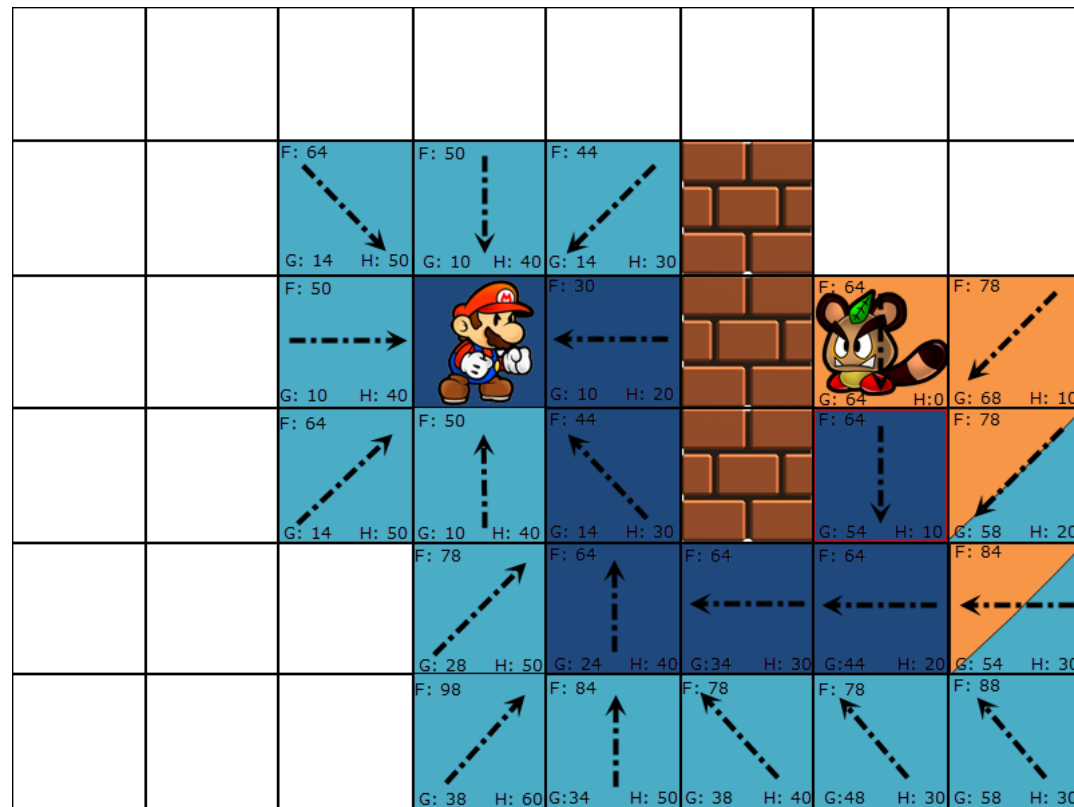
{ Eksempel

- noden med laveste F værdi vælges og flyttes fra den åbne til den lukkede liste



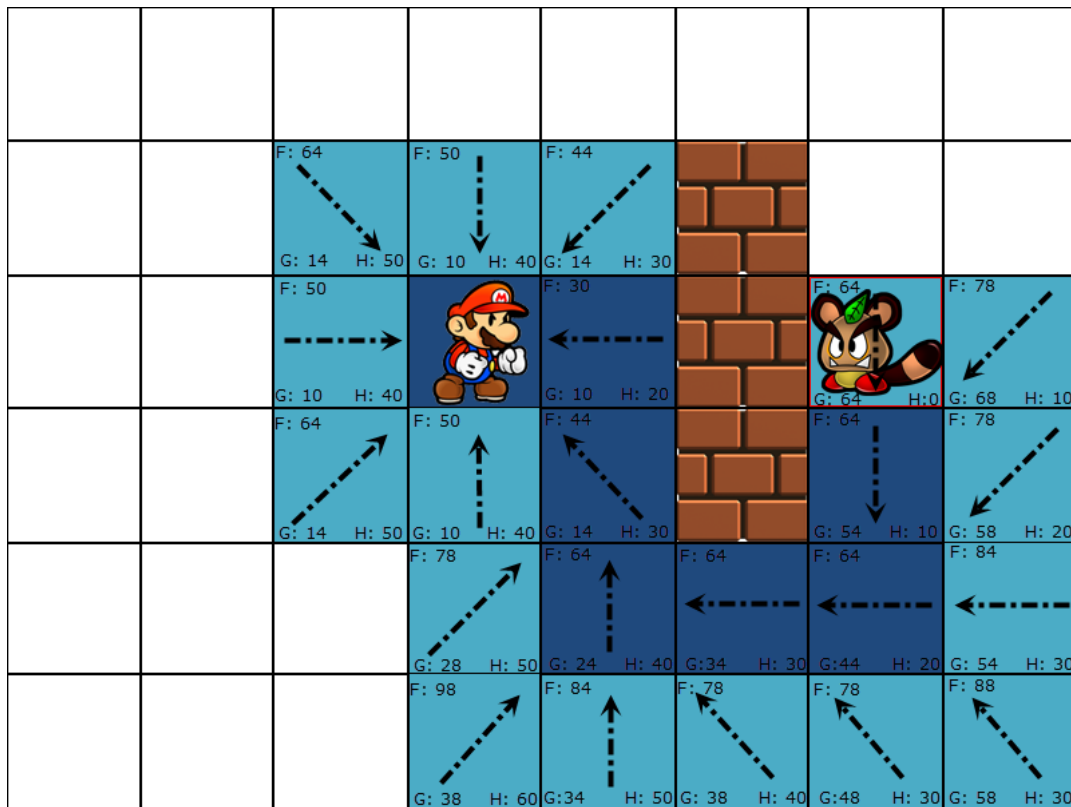
Eksempel

- Naboer undersøges
 - Parents og G,H,F værdier sættes, alternativ G score beregnes på naboer, som allerede var på listen



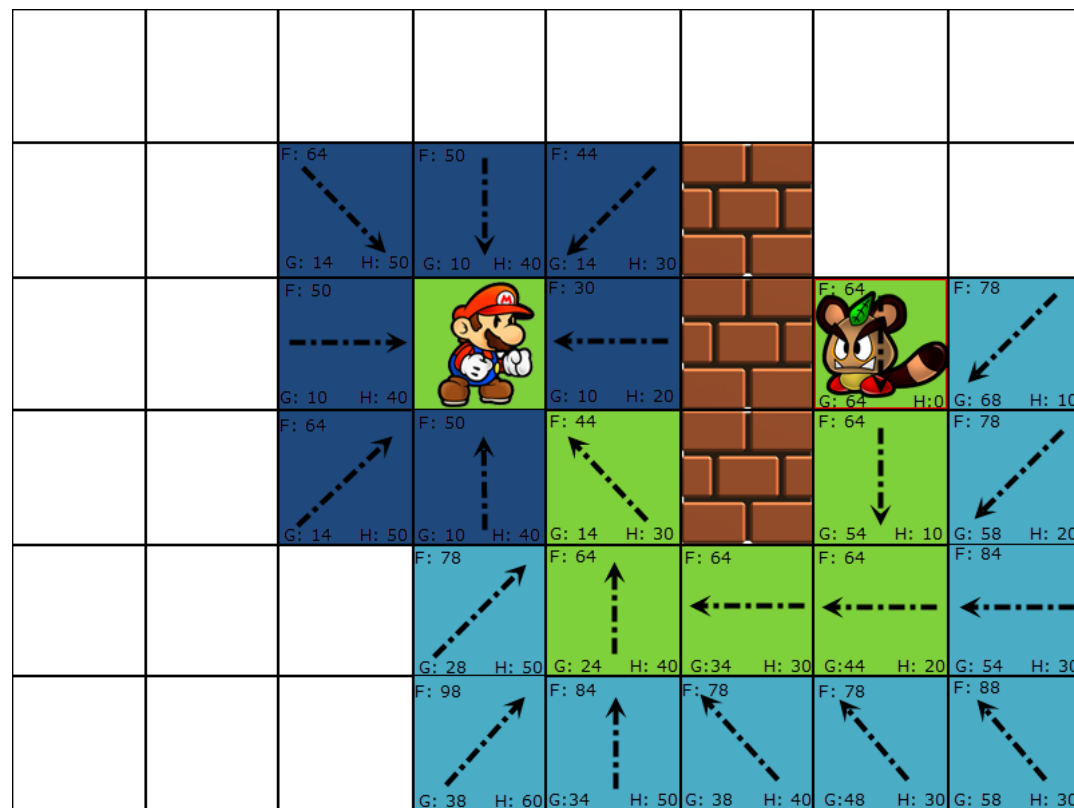
{ Eksempel

- noden med laveste F værdi vælges og flyttes fra den åbne til den lukkede liste
→ Målet er fundet



{ Eksempel

- Stien findes nu ved at følge parent-child forholdene



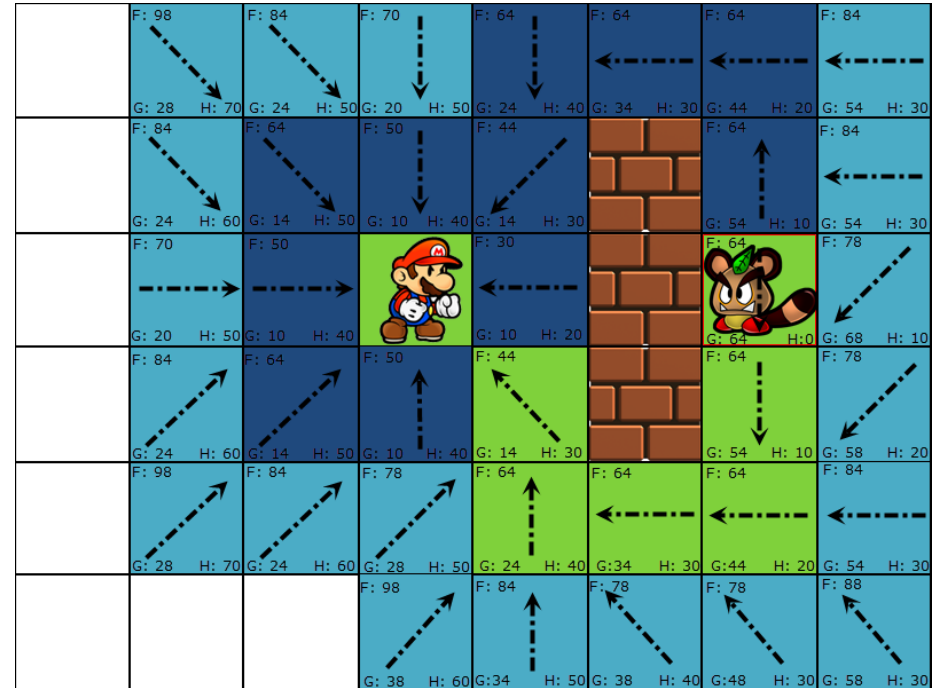
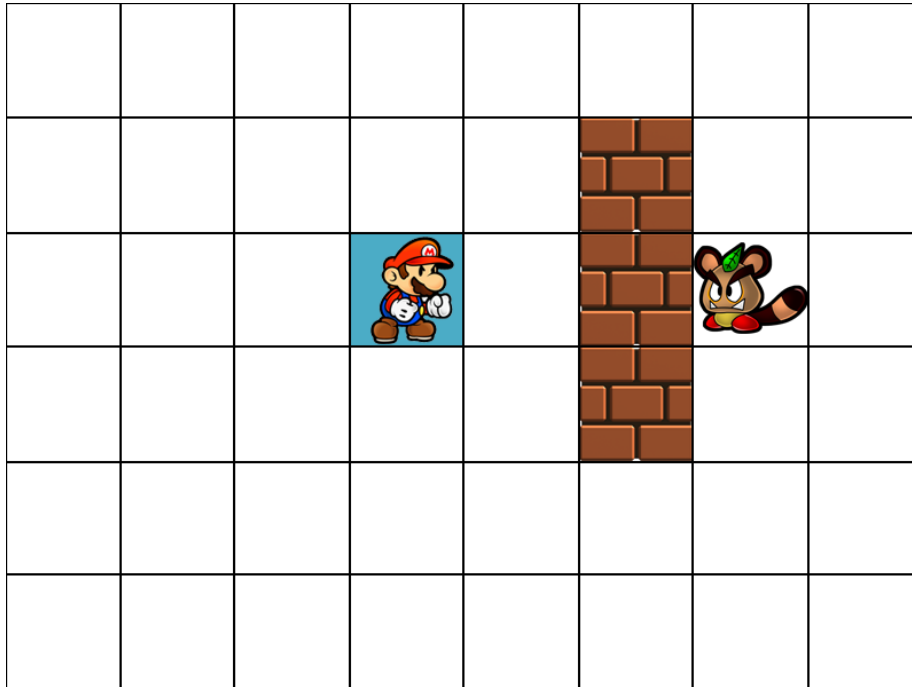
Eksempel

	F: 98 G: 28 H: 70 ↘	F: 84 G: 24 H: 50 ↘	F: 70 G: 20 H: 50 ↓	F: 64 G: 24 H: 40 ↓	F: 64 G: 34 H: 30 ←	F: 64 G: 44 H: 20 ←	F: 84 G: 54 H: 30 ←
	F: 84 G: 24 H: 60 ↘	F: 64 G: 14 H: 50 ↘	F: 50 G: 10 H: 40 ↓	F: 44 G: 14 H: 30 ↙	Brick Wall	F: 64 G: 54 H: 10 ↑	F: 84 G: 54 H: 30 ←
	F: 70 G: 20 H: 50 →	F: 50 G: 10 H: 40 →		F: 30 G: 10 H: 20 ←	Brick Wall	F: 64 G: 64 H: 0 ↘	F: 78 G: 68 H: 10 ↘
	F: 84 G: 24 H: 60 ↗	F: 64 G: 14 H: 50 ↗	F: 50 G: 10 H: 40 ↑	F: 44 G: 14 H: 30 ↙	Brick Wall	F: 64 G: 54 H: 10 ↓	F: 78 G: 58 H: 20 ↘
	F: 98 G: 28 H: 70 ↗	F: 84 G: 24 H: 60 ↗	F: 78 G: 28 H: 50 ↗	F: 64 G: 24 H: 40 ↑	F: 64 G: 34 H: 30 ←	F: 64 G: 44 H: 20 ←	F: 84 G: 54 H: 30 ←
			F: 98 G: 38 H: 60 ↗	F: 84 G: 34 H: 50 ↑	F: 78 G: 38 H: 40 ↙	F: 78 G: 48 H: 30 ↙	F: 88 G: 58 H: 30 ↙



{ Eksempel

Herunder ses starten samt slutresultatet



Opsummering af AStar

1. Placer start noden på den åbne liste
2. Gentag følgende
 - a) Find noden med den laveste F værdi på den åbne liste (den valgte node)
 - b) Flyt noden til den lukkede liste
 - c) Gør følgende for alle nabonoder til den valgte node
 - Hvis noden ikke er walkable eller er på den lukkede liste, ignoreres den
 - Hvis den ikke er på den åbne liste tilføjes den til listen. Den valgte node gøres til parent og F,G og H værdierne beregnes
 - Hvis noden allerede er på listen kontrolleres det om stien gennem den nuværende node er bedre, ved at sammenligne G værdierne. Hvis G er lavere er stien bedre. Hvis dette er tilfældet ændres nodens parent til den nuværende node, G,H og F scores genberegnes
 - d) Søgningen stoppes når
 - Målet er tilføjet til den lukkede liste dvs. at en sti blev fundet
 - Den åbne liste er tom dvs. at der ikke blev fundet en sti
3. Stien findes ved at arbejde sig tilbage fra målet gennem parent noder til startknuden er nået



Links til Astar

Wikipedia:

[http://en.wikipedia.org/wiki/A* search algorithm](http://en.wikipedia.org/wiki/A*_search_algorithm)

A* pathfinding for beginners:

<https://csis.pace.edu/~benjamin/teaching/cs627/webfiles/Astar.pdf>



{ Øvelse

Lav en implementering af AStar algoritmen

- I kan eventuelt benytte jer af det MonoGame projekt, som ligger på moodle. Projektet indeholder et grid af noder og et interface :0
 - Giver jer tid til at fokusere på at få algoritmen til at virke

